

RoboCompiler: Graph-Native Compilation of Closed-Chain Robots for Consistent Modeling, Control, and Simulation

Mehdi Heydari Shahna , Joongheon Kim , and Jouni Mattila 

Abstract—Robots with kinematic loops, coupled actuators, and changing contacts require consistent models of configuration, motion, force, and dynamics. Yet these interfaces are often reconstructed separately for control and simulation, making closure and actuation consistency difficult to maintain. This paper presents RoboCompiler, a graph-native framework that compiles a canonical mechanism graph into a shared mechanical interface. From bodies, joints, frames, inertias, and actuator ports, it constructs closure paths and analytic residual Jacobians, then assembles feasible configurations through rank-checked continuation and correction. A tangent lift maps independent velocities to full robot and task motion, while paired actuator-port maps preserve virtual work. A constraint-curvature correction extends the reduction to accelerations and projected rigid-body dynamics, including floating-base and support modes. Cycle-local evaluation, generated Jacobians, and dependency-aware reuse enable localized updates when closure inputs change. We evaluate physical loops and task-induced constraints on a Komatsu excavator, Unitree Go2, Franka Panda, Kangaroo, and a six-UPS Stewart platform. High-precision constrained-dynamics and independent Pinocchio checks confirm mechanical consistency; MuJoCo and Isaac Sim/PhysX executions demonstrate task performance and model reuse under native contact. For Kangaroo, compilation reduces residual-and-Jacobian evaluation time by 96.7% and closed-loop rollout wall time by 66.8%, with dynamics and control held fixed.

Index Terms—Closed-chain mechanisms, constrained dynamics, kinematics, parallel robots, robot modeling, simulation.

I. INTRODUCTION

A robot’s physical joints and actuator attachments determine its admissible motion and how mechanical work enters the system. Closed linkages make these relationships difficult to carry through a computational model: a dynamics library may open a loop to form a tree, a controller may use a different set of independent coordinates, and a simulator may impose the same connection through native constraints. Contact and support introduce further mode-dependent constraints. These representations can describe one mechanism, but their configuration, velocity, acceleration, and effort interfaces must remain compatible. This requirement is especially consequential for robots with multiple coupled linkages, moving actuator bodies, and floating bases [1].

This work was supported by the grant named the Post Docs in Companies (PoDoCo) program in Finland.

M. H. Shahna and J. Mattila are with the Faculty of Engineering and Natural Sciences, Tampere University, Tampere, Finland; and J. Kim is with the Department of Electrical and Computer Engineering at Korea University, Seoul, Republic of Korea. (Corresponding author e-mail: mehdi.heydarishahna@tuni.fi)

Code: <https://github.com/Mehdi-Heydari-Shahna/RoboCompiler>

Robot-description methods provide a foundation for representing such systems. URDF+ adds loop joints and coupling relations to URDF and automatically groups constrained bodies for recursive algorithms [2]. Extended URDF records closure and actuation information alongside a tree model and provides tools to construct corresponding Pinocchio models [3]. Both make physical connections available to downstream computation. A separate line of work infers a robot’s connectivity and joint geometry from proprioceptive data [4].

Methods for operating on constrained configurations address another part of the problem. Manifold-based kinodynamic planning integrates motion on holonomic constraint manifolds [5]. HyRoDyn combines analytical submechanism solutions with numerical closure and provides mappings between independent and actuation coordinates for kinematics and dynamics [6], [7]. The Kangaroo model develops detailed closed-loop kinematic and actuator relationships for a floating-base mechanism [8]. PACDM constructs closure residuals and analytical Jacobians from ordered transformation paths, then uses rank selection, defect homotopy, and continuation to obtain a feasible configuration and its local differential map [9].

Constrained dynamics and control have advanced in parallel. Constraint embedding and GRBDA enable recursive dynamics for mechanisms with local loops [10], [11]; recent work develops dynamics derivatives for constraint-embedded models [12]. Pinocchio provides constrained-dynamics solvers, including recursive closed-loop forward dynamics [13]–[15], while MuJoCo represents loop connections through equality constraints and executes contact-rich simulations [16], [17]. At the control level, hierarchical optimization, redundancy resolution, geometric motion policies, and optimal control provide ways to use constrained kinematics and dynamics for tasks [18]–[23].

Table I compares selected methods by the interfaces they expose, including closed-chain modeling, assembly, kinematic mapping, dynamics, floating-base contact, and multi-simulator execution. The comparison is intended to summarize documented interface coverage rather than to rank complete software systems with different design goals.

Taken together, these advances address major parts of closed-chain model description, constrained assembly, kinematic reduction, dynamics, simulation, and control. They leave, however, an important integration question: how can these computational views be generated from the same physical mechanism description while preserving assembly, mo-

TABLE I

CAPABILITIES OF CLOSED-CHAIN MODELING AND DYNAMICS TOOLS. ✓: YES; ○: PARTIAL; ×: NO; -: NOT STATED IN THE PRIMARY SOURCE; N/A: NOT APPLICABLE. LOOPS IN MODEL: CLOSURES ARE PART OF THE MODEL/GRAPH DESCRIPTION. AUTO. CUT SEL.: SPANNING TREE AND CUT JOINTS CHOSEN AUTOMATICALLY FROM A GENERAL BODY-JOINT GRAPH. ASSEMBLY: BUILT-IN POSITION-LEVEL CLOSURE SOLVE. TANGENT MAP: EXPLICIT MAP FROM INDEPENDENT TO FULL VELOCITIES. PORT MAPS: EXPLICIT ACTUATOR-PORT VELOCITY MAP WITH ITS DUAL (VIRTUAL-WORK-CONSISTENT) EFFORT MAP; ○ = JOINT-SPACE TORQUES, GEARS OR COUPLINGS ONLY. FD: R = REDUCED/PROJECTED, K = LAGRANGE-MULTIPLIER/KKT, REC = RECURSIVE, S = SOFT (REGULARIZED) CONSTRAINTS; A/N = ANALYTIC/NUMERICAL $\dot{J}\dot{q}$. FLOAT. BASE + CONTACT: FLOATING-BASE MODELS WITH CONTACT OR SUPPORT CONSTRAINTS (BILATERAL OR UNILATERAL). MULTI-SIM.: EXECUTION OR VALIDATION OF THE SAME MODEL IN EXTERNAL SIMULATORS OR DYNAMICS LIBRARIES (✓: SEVERAL; ○: ONE).

Work	Loops in model	Auto. cut sel.	Closure types	Assembly	Tangent map	$\dot{J}\dot{q}$ term	Port maps	FD	Float. base + contact	Multi-sim.
Bordalba et al. [5]	–	–	holonomic $\Phi(q) = 0$	✓	○ ^a	–	–	manifold ODE	– ^b	–
HyRoDyn [6], [7]	○	×	submechanism library ^c	✓	✓	✓ (A)	✓	R	–	–
Kangaroo model [8]	✓	×	relative pose, selected axes	✓	✓	✓ (A)	✓	ID only ^d	✓	○
URDF+ [2]	✓	×	URDF joint types, linear couplings	–	○	–	○	n/a ^e	–	–
Constraint embedding / GRBDA [10], [11]	✓ ^f	×	explicit G , implicit $\phi(q)$	○	✓	✓ (A)	○	Rec	○	○
LCABA [14] / Pinocchio 4.1 [13], [15]	○	○ ^g	point (3D), weld (6D)	○ ^h	×	✓ (A) ⁱ	○	K, Rec	✓	○ ^j
MuJoCo 3.3.7 / 3.14 [16], [17]	✓	×	point, weld, scalar joint/tendon	○ ^k	×	× / ○ ^l	✓	S	✓	n/a
PACDM [9]	○	×	SE(3) path-pose equality	✓	✓	–	–	× (kinematics)	–	○
RoboCompiler (this work)	✓	○ ^m	point, revolute, universal; welds	✓	✓	✓ (N) ⁿ	✓	R	✓	✓

^a Orthonormal basis of the state-manifold tangent space at each chart centre (local chart coordinates), not an explicit $\dot{q}$ map.

^b All examples are fixed-base; unilateral contact is listed as future work.

^c Closed-form submechanism types (e.g., 1-RRPR, 2-SPU+1U, 6-UPS) plus a numerical fallback; users annotate the submechanisms.

^d Inverse dynamics via Lagrange multipliers; no forward-dynamics algorithm is stated.

^e Model format; its output is intended for recursive constraint-embedding algorithms.

^f Through URDF+ in the GRBDA library ($\geq v2.1.0$); the T-RO paper itself uses a cluster-graph formalism.

^g The SDF parser keeps one parent joint per multi-parent link in the tree and converts the others into constraints.

^h No library routine; the official examples hand-code an inverse-geometry loop.

ⁱ Analytic drift $\gamma_{c,T}$. In Pinocchio 4.1.0 `lcaba`, the Baumgarte term is active only for 6D constraints.

^j A Pinocchio unit test compares closed-chain MJCF kinematics with MuJoCo; the LCABA benchmarks stay inside Pinocchio.

^k No assembly routine. With anchors, the constraint is assumed satisfied at `qpos0`; with sites, the sites snap together at the beginning of simulation.

^l Not computed in 3.3.7; added for `connect/weld` in 3.7.0; still absent for the `tendon` rows in our 3.14.0 tests.

^m Tree and revolute cuts generated from physical records in the excavator compiler; the Stewart compiler generates the physical tree for each spherical tree-joint choice; the Kangaroo compiler takes declared cuts.

ⁿ Analytic closure Jacobian; $\dot{J}\dot{q}$ by a centered directional finite difference of it.

tion, acceleration, actuation, and mechanical-power consistency across them?

A physical description, a feasible closure solution, and a constrained-dynamics solver must refer to the same assembly branch and actuator coordinates if their outputs are to be used together. Changing the computational tree must preserve the restrictions imposed by each physical connection. Changing independent coordinates must carry the task differentials and reduced inertia with them. Actuator velocities and efforts must transform as a power-conjugate pair, and the acceleration map must retain constraint curvature. Agreement of coordinate vectors or successful model import alone does not establish all of these relationships.

We develop RoboCompiler to address this integration problem. RoboCompiler is a graph-native compiler built around a canonical mechanism graph (CMG) whose body, joint, frame, inertia, and actuator-port records retain their physical identities. Starting from the physical mechanism records, the compiler selects tree and cut assignments or accepts declared cuts,

generates ordered closure paths and coordinate-dependency records, and provides the path structure required by PACDM.

PACDM continuation and correction establish a feasible assembly branch, after which rank checks and analytical closure Jacobians provide a tangent lift from independent to full velocities. A constraint-curvature term extends the lift to accelerations, while paired port-velocity and effort maps preserve virtual work. These quantities feed reduced rigid-body dynamics and task-level computation, and backend-specific views support independent dynamics checks and native execution in external simulators. Cycle-local evaluation and reuse of unaffected dependency modules reduce repeated closure computation while preserving the accepted PACDM solution.

In this sense, RoboCompiler is complementary to robot-description formats, constrained-dynamics libraries, and simulators: rather than replacing them, it compiles a shared physical mechanism description into mechanically consistent interfaces that these tools can use. Table I and Figs. 1 and 2

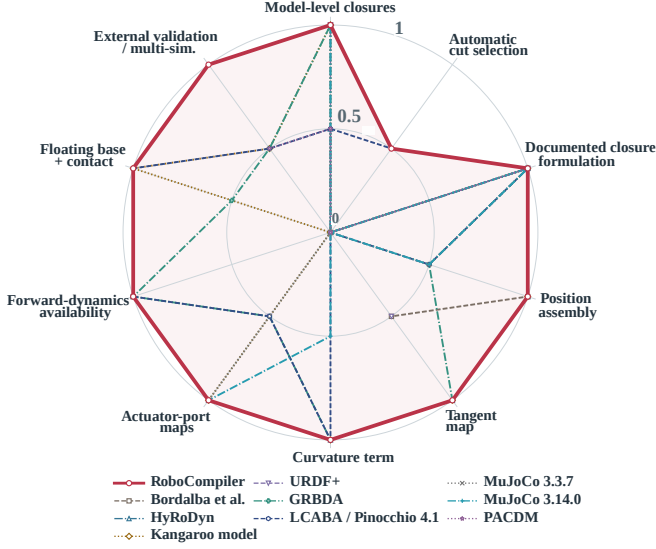


Fig. 1. Categorical interface coverage from Table I, with RoboCompiler highlighted. Scores 1, 0.5, and 0 encode documented, partial, and absent support; not-stated and inapplicable categories remain gaps.

summarize the resulting interface coverage: among the compared methods, only RoboCompiler is documented to combine point, revolute, and universal cuts and position-level assembly with an explicit independent-to-full tangent map, virtual-work-dual actuator-port maps, reduced forward dynamics, floating-base/contact workflows, and multi-simulator validation.

The contributions of this work are as follows:

- We introduce a graph-native compilation procedure that transforms physical mechanism and task records into ordered closure paths, dependency modules, rank-checked coordinate partitions, and connected interfaces for configuration, motion, actuation, and dynamics.
- From a shared assembled configuration, we generate the tangent lift, a constraint-curvature correction, and virtual-work-dual actuator-port maps. Together, these form a common independent-coordinate interface for motion control and reduced inverse and forward dynamics of fixed-base robots and of floating-base robots under ideal support modes.
- We generate cycle-local analytical closure Jacobians and dependency-aware update schedules from the graph. Selective evaluation and reuse reduce repeated work during continuation and local task changes while preserving the underlying PACDM closure and correction equations.
- We evaluate the interfaces on three physically closed-chain mechanisms and two robots with task or actuator coupling. Independent constrained-dynamics checks, matched computational ablations, and executions in MuJoCo, Pinocchio, and Isaac Sim demonstrate mechanical consistency, computational benefit, and reuse across environments.

The complete code for all five robots and their MuJoCo, Pinocchio, and Isaac Sim/PhysX workflows is available open source at <https://github.com/Mehdi-Heydari-Shahna/RoboCompiler>.

II. PHYSICAL DESCRIPTION AND COMPILATION

A. Physical Data and Velocity Conventions

Let $\mathcal{G} = (\mathcal{B}, \mathcal{J})$ be a body–joint multigraph (principal quantities are listed in Table II). Each vertex denotes a rigid-body instance with geometry, attachment frames, and inertial parameters. Each edge denotes a physical joint with a permitted motion model and limits. Repeated parts remain separate instances. Actuator ports, transmissions, and their limits are separate physical records, so the choice of independent coordinates is decoupled from actuation.

For an oriented joint $j = (b, c)$, define

$$X_j(q_j) = A_j Q_j(q_j) B_j^{-1} \in \text{SE}(3), \quad (1)$$

where $A_j = {}^bT_{J_b}$ and $B_j = {}^cT_{J_c}$ locate the two attachment frames, and $Q_j = {}^J_bT_{J_c}$ describes the allowed relative motion. Thus $X_j = {}^bT_c$ maps coordinates from frame c to frame b . Reverse traversal uses X_j^{-1} . Fixed, revolute, prismatic, spherical, and universal joints fit this description when their local parameterizations and motion maps are supplied. Additional holonomic couplings, for example a prescribed gear relation, enter as $\gamma(q) = 0$, with their equations and derivatives provided as physical inputs.

The augmented configuration $q \in \mathcal{Q}$ contains all joint coordinates, including those on cut edges, and any free-root poses. Its generalized velocity is $v \in R^n$, with

$$\dot{q} = N(q)v. \quad (2)$$

Here N is a full-column-rank local map from generalized velocities to configuration derivatives; in particular, a quaternion derivative is not an angular velocity. Spatial vectors use angular-first ordering, and Ad_T is the corresponding adjoint. A numerical increment δv is applied through a manifold integration operation $q \oplus \delta v$.

TABLE II
PRINCIPAL QUANTITIES AND THEIR ROLES.

Symbol	Definition
$q \in \mathcal{Q}, v \in R^n$	Augmented configuration and generalized velocity
$\rho \in R^s, C$	Full closure residual; $C = D\rho N$
$r, d = n - r$	Closure rank and independent velocity dimension
$u \in R^d, E$	Independent speeds and lift $v = Eu$
v_T, E_T	Tree speeds and their lift $v_T = E_T u$
x, J	Task coordinates and reduced task Jacobian
ℓ, L, f	Actuator displacement, reduced transmission, effort
M_r, h_r	Reduced inertia and bias terms
W, g	Reduced physical metric and objective differential

B. From Physical Edges to Closure Comparisons

Choose a spanning forest $\mathcal{T}$ and a reference body for each connected component. A computational reference does not impose a physical fixation: a free-root pose remains a state variable. The number of non-tree edges is

$$\mu = |\mathcal{J}| - |\mathcal{B}| + \kappa, \quad (3)$$

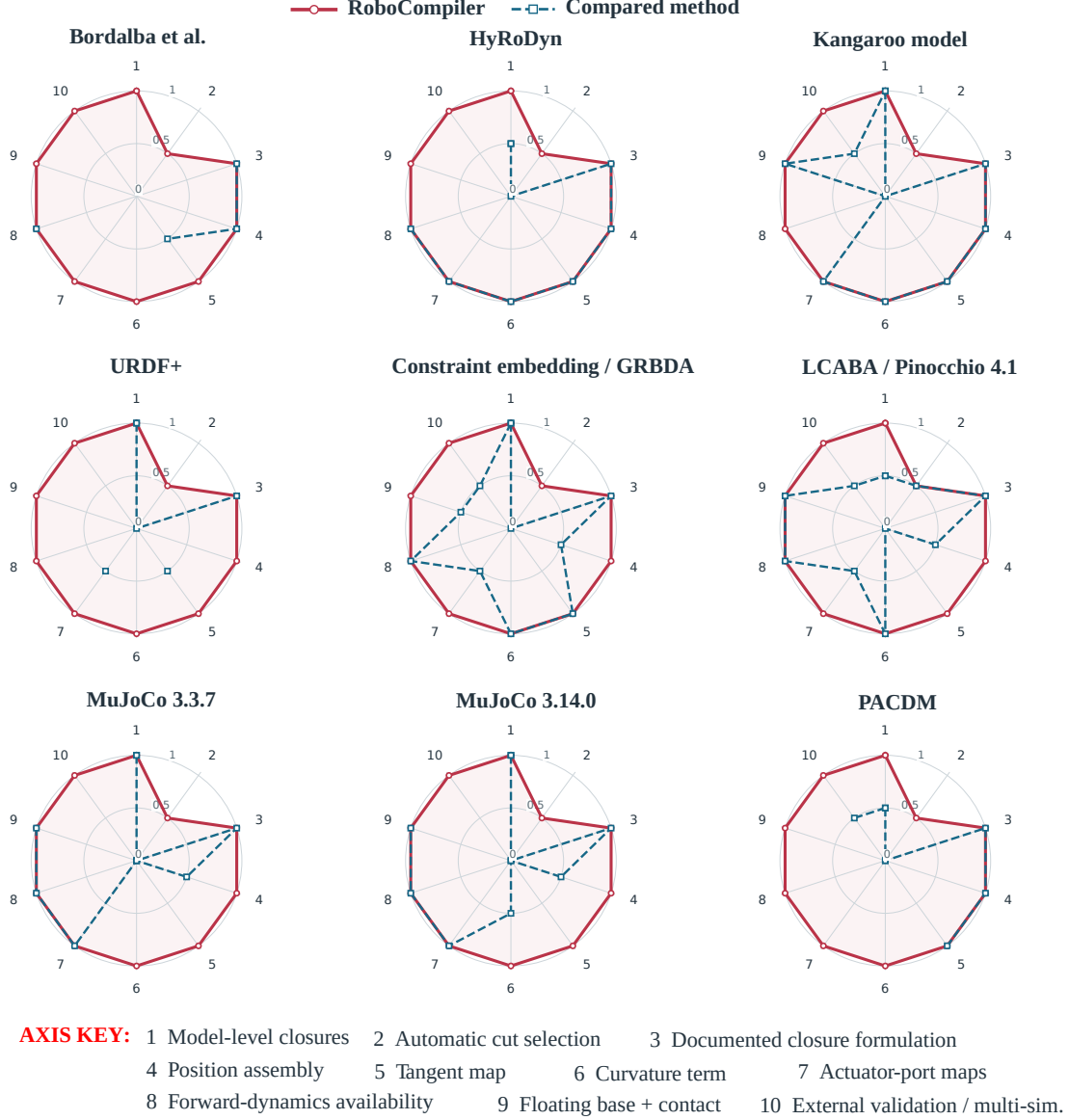


Fig. 2. Pairwise views of the same categorical entries in Table I: RoboCompiler (solid red) and each listed method (dashed blue). A missing or inapplicable entry is unscored.

where κ is the number of connected components. This cycle rank is not the number of independent scalar constraints.

Each chord $e = (b, c)$ defines a unique ordered forest path P_e between its endpoints. Let $\sigma_{ej} \in \{-1, 1\}$ specify traversal direction. Form

$$T_e^+ = \prod_{j \in P_e}^{\rightarrow} X_j^{\sigma_{ej}}, \quad T_e^- = X_e, \quad \Delta_e = (T_e^-)^{-1} T_e^+. \quad (4)$$

Closure requires $\Delta_e = I$. Retaining q_e allows the cut joint to execute its physical relative motion. Replacing this equality by a weld after deleting q_e would generally change the mechanism. Joint-specific constraint elimination is possible, but it must preserve the same admissible relative transformations.

Completeness of the edge construction. For fixed component-root poses and specified joint configurations,

forest propagation together with $\Delta_e = I$ for every chord satisfies every physical edge relation in $\mathcal{G}$.

Proof. Forest propagation assigns body poses satisfying all tree edges. For a chord, $\Delta_e = I$ equates the propagated endpoint transform with its physical joint transform. Every edge is either a tree edge or a chord. Conversely, body poses satisfying every edge necessarily satisfy these comparisons.

This statement establishes structural edge completeness. Feasibility, including the supplied couplings $\gamma(q) = 0$, is established by the closure solve of Section III, and scalar independence by its rank analysis. Figure 3 summarizes the construction.

C. Dependency Modules and Compiler Output

A candidate local partition distinguishes independent and dependent variables. For each closure comparison, let $\mathcal{D}_e$

contain the dependent variable blocks occurring in its ordered factors. The supplied coupling residuals are included through their own dependent supports. An inverted variable index connects residual blocks sharing a dependent variable; its connected components define candidate modules. Modules may share prescribed independent variables. Their dependent variables are disjoint, permitting separate closure solves when those prescribed values are fixed.

This grouping is conservative because structural dependencies can cancel algebraically. An independent-only constraint is retained: it must be locally redundant, or the partition is revised. Likewise, an unconstrained variable is never assigned to the dependent set. The rank conditions in Section III decide whether the candidate partition yields a valid reduction.

Each emitted comparison records ordered factors, traversal signs, coordinate indices, and source-edge identifiers. Different comparisons in a dependency module may have different endpoints. PACDM is therefore applied to each endpoint-matched pair and then assembled in common module coordinates. When a module forms a single common-endpoint path family, N_p paths need only $N_p - 1$ pairwise comparisons [9].

Forest construction and explicit path extraction require $O(|\mathcal{B}| + |\mathcal{J}| + L_P)$ operations, where $L_P = \sum_e (|P_e| + 1)$. With union-find, grouping N_c closure/coupling blocks costs $O(N_c + S_D \alpha(N_c))$, where S_D is their total dependent-support size. These bounds cover the structural compilation stage; numerical rank analysis and closure resolution (Section III) are separate stages. Listing 1 shows the Kangaroo input. It contains only physical records; the compiler rejects inputs that pre-author paths, partitions, modules, or coordinate orders, and generates them instead. For Kangaroo, it yields 76 joint and 64 chart coordinates in six loop modules. Each module is corrected separately and is skipped when its motor inputs are unchanged. Function and record names are those of the implementation; variable names follow the notation of this paper.

III. PACDM CLOSURE AND LOCAL REDUCTION

A. Path-Assembled Differentials

Following PACDM [9], define

$$\begin{aligned} \rho_e(q) &= \text{Log}(\Delta_e(q))^\vee, & \rho(q) &= \text{col}(\text{col}_e \rho_e(q), \gamma(q)), \\ C(q) &= D\rho(q)N(q). \end{aligned} \quad (5)$$

The logarithm is evaluated on a fixed local branch whose domain is enforced as an acceptance condition (Section III-B). The feasible configuration set is $\mathcal{C} = \{q : \rho(q) = 0\}$.

For an ordered path $T^\sigma = F_1^\sigma \cdots F_k^\sigma$, define $Z_i^\sigma = F_1^\sigma \cdots F_{i-1}^\sigma$. Local motion maps Ξ_i^σ satisfy $(\dot{F}_i^\sigma (F_i^\sigma)^{-1})^\vee = \Xi_i^\sigma v$. Product differentiation gives

$$(\dot{T}^\sigma (T^\sigma)^{-1})^\vee = H^\sigma v, \quad H^\sigma = \sum_i \text{Ad}_{Z_i^\sigma} \Xi_i^\sigma. \quad (6)$$

Consequently, the differential of each residual is

$$C_e = J_l^{-1}(\rho_e) \text{Ad}_{(T_e^-)^{-1}}(H_e^+ - H_e^-), \quad (7)$$

where J_l is defined by $(\dot{\Delta} \Delta^{-1})^\vee = J_l(\rho) \dot{\rho}$. Equation (7) retains the logarithm differential away from closure; substituting

```
physical = { # Kangaroo records, abridged
  "floating_root": "base_link",
  "bodies": [...], # 78, with mass and inertia
  "joints": [...], # 77 edges, frames, limits
  "loop_cuts": [ # 16 point + 8 universal
    {"id": "left_hip_yaw_closed",
     "type": "point_coincidence",
     "body1": "left_yaw_rod",
     "body2": "left_hip_yaw", ...},
    {"id": "left_hip_differential_2a",
     "type": "universal", ...}, ...],
  "actuators": [ # 12 actuator ports
    {"id": "leg_left_1_actuator_torque",
     "joint_id": "leg_left_1_motor",
     "gear": 1.0,
     "force_bounds_N": [-2000.0, 2000.0]}, ...],
  "drive": {...}, "sole_sites": [...]}
```

```
comp = compile_graph(physical)
# generated: tree, cut paths, charts, 6 modules
solver = ModularSolver(comp, x0) # x0: assembled
q, E, info = solver.step(q_motor) # 12 motors
# E (joint rows); modules whose inputs are
# unchanged are skipped
```

Listing 1. Abridged Kangaroo input and compiler calls. The input holds physical records only; step returns the joint configuration and the joint rows of the lift E in (9).

$J_l^{-1} = I$ there is generally incorrect. Rigid factors contribute through the adjoint transports even when their own derivatives vanish. The supplied coupling rows contribute $D\gamma(q)N(q)$ to C .

B. Rank, Recovery, and Continuation

Consider a neighborhood of a feasible configuration where $\text{rank } C = r$ is constant. A permutation Π partitions the velocity into $\Pi v = \text{col}(v_a, v_p)$, with $v_p \in \mathbb{R}^r$. Choose a fixed row selector $S \in \mathbb{R}^{r \times s}$ such that

$$\begin{aligned} C\Pi^\top &= [C_a \ C_p], & \det R_p &\neq 0, \\ R_a &= SC_a, & R_p &= SC_p. \end{aligned} \quad (8)$$

The selected rows must span the full row space locally. Then

$$v = E(q)u, \quad E = \Pi^\top \begin{bmatrix} I_d \\ -R_p^{-1}R_a \end{bmatrix}, \quad CE = 0, \quad (9)$$

with $u = v_a$ and $d = n - r$. These are independent speeds, not necessarily derivatives of a global coordinate vector. The corresponding position solve is performed in compatible local configuration charts.

Rank-revealing factorizations select rows and candidate dependent blocks using declared length and velocity scales. Linear systems are solved without explicitly forming R_p^{-1} . A singular R_p with unchanged rank C calls for a new velocity partition, that is, a computational reconfiguration (Section V-C); a change in rank C itself is a distinct, mechanism-level event. The complete residual is checked after every accepted solve.

For acquisition, PACDM uses an artificial defect [9]. Given q^0 , set

$$\begin{aligned} D_e(\eta) &= \text{Exp}((1 - \eta) \text{Log } \Delta_e(q^0)), \\ \rho_e^\eta(q) &= \text{Log}(D_e(\eta)^{-1} \Delta_e(q))^\vee, & \eta &\in [0, 1]. \end{aligned} \quad (10)$$

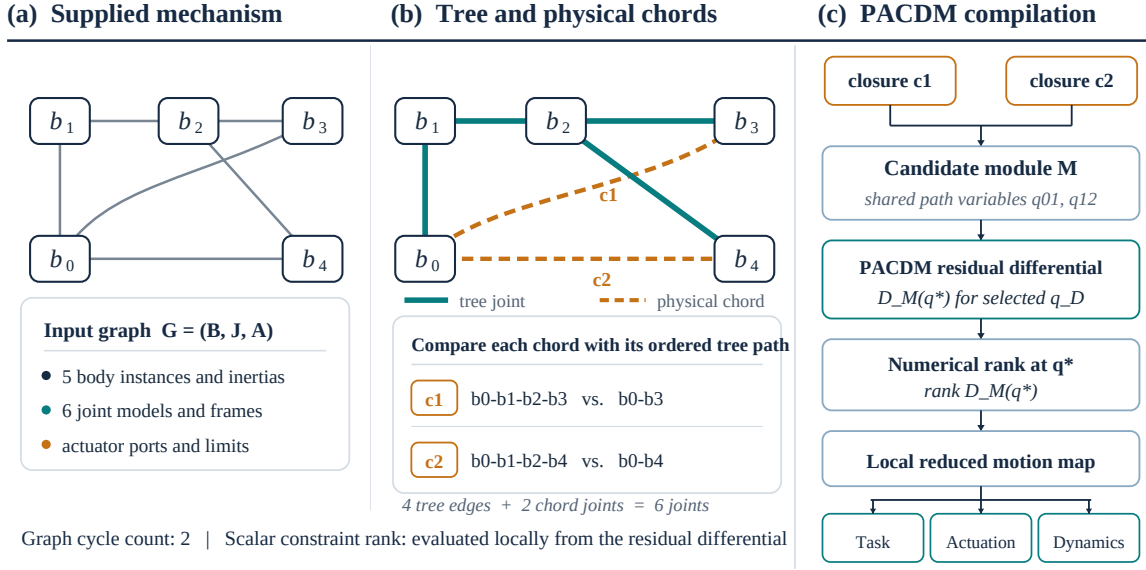


Fig. 3. Compilation from a supplied physical mechanism graph. (a) Body instances, joint attachment frames, and actuator ports define the input. (b) A five-body, six-joint connected graph admits a four-edge spanning tree and two chords; each chord is compared with its ordered tree path. (c) Comparisons sharing dependent variables form a candidate module. PACDM supplies the residual differential, while numerical rank determines the local reduction. Dashed chords remain physical joints. Graph cycle count and scalar constraint rank are distinct.

Coupling residuals use $\gamma^\eta(q) = \gamma(q) - (1-\eta)\gamma(q^0)$. The initial state then satisfies the artificial problem at $\eta = 0$. Continuation removes the defect while prescribed independent variables are held fixed. Acquisition uses its own independent row set and requires a regular feasible continuation path. At $\eta = 1$, all physical residuals are checked and the physical reduction is recomputed; acquisition and physical row selectors need not coincide.

During motion, the lift predicts $q_{k+1}^- = q_k \oplus (hE(q_k)u_k)$, followed by a local closure correction. Recovery is invoked when direct correction fails. Joint limits, the selected assembly branch, and the logarithm domain remain acceptance conditions.

IV. MOTION, ACTUATION, AND DYNAMICS INTERFACES

A. Task and Actuator Maps

For a task $x = f_x(q) \in \mathbb{R}^m$ expressed in a fixed local task chart, let $J_x = Df_x N$. A differentiable actuator displacement map $\ell = f_\ell(q) \in \mathbb{R}^{n_f}$ similarly gives $J_\ell = Df_\ell N$. The compiled interfaces are

$$\dot{x} = Ju, \quad J = J_x E, \quad \dot{\ell} = Lu, \quad L = J_\ell E. \quad (11)$$

Actuator displacement can be a cylinder extension or a local motor angle. Transmissions without a holonomic displacement map are included through an explicitly supplied velocity map and its effort dual.

If $f \in \mathbb{R}^{n_f}$ is the actual effort conjugate to $\dot{\ell}$, virtual work gives

$$\tau_r = L^\top f, \quad u^\top \tau_r = \dot{\ell}^\top f. \quad (12)$$

A commanded effort and an actual effort are different quantities when drive dynamics are present. Such dynamics enter through specified states z and commands c , for example $\dot{z} = f_z(z, c, q, v)$ and $f = f_f(z, c, q, v)$. These constitutive

laws are supplied as model inputs alongside the mechanism graph. Friction and transmission losses enter as additional terms; Eq. (12) holds at the declared mechanical ports.

B. Physical Inertia and Reduced Dynamics

Closure coordinates attached to cut joints do not create additional bodies or mass. Let $q_T = \pi_T(q)$ denote the tree configuration and let $v_T = K_T(q)v$ be its differential velocity map. With N_T denoting the tree configuration-rate map, compatibility requires $D\pi_T(q)N(q) = N_T(q_T)K_T(q)$. Define $E_T = K_T E$. The tree model contains each physical body's inertia exactly once. Its applied effort is decomposed as $\tau_T = \tau_{a,T} + \tau_{\text{ext},T}$, with $E_T^\top \tau_{a,T} = L^\top f$ and $\tau_{r,\text{ext}} = E_T^\top \tau_{\text{ext},T}$. Its constrained equations are

$$M_T \dot{v}_T + h_T = \tau_T + J_{c,T}^\top \lambda, \quad J_{c,T} E_T = 0, \quad (13)$$

where λ represents ideal internal closure reactions and h_T includes the tree's bias terms [24]. Since $v_T = E_T u$, projection yields

$$\begin{aligned} M_r \dot{u} + h_r &= L^\top f + \tau_{r,\text{ext}}, \\ M_r &= E_T^\top M_T E_T, \\ h_r &= E_T^\top (h_T + M_T \dot{E}_T u). \end{aligned} \quad (14)$$

The derivative $\dot{E}_T$ is taken along the physical trajectory and includes any variation of K_T . Omitting $M_T \dot{E}_T u$ changes the acceleration model. External forces, including contact forces when present, remain in $\tau_{r,\text{ext}}$ rather than disappearing with internal loop reactions.

Cut-joint actuator efforts are applied through physical attachment wrenches or an equivalent local transmission, so every effort acts on retained physical coordinates.

Positive definiteness of M_r requires a positive physical inertia on the image of E_T and full column rank of that

```

physical = { # Panda records, abridged
  "joints": [...], # 11 tree edges, 9 moving
  "affine_couplings": [{"master": "finger_joint1",
    "slave": "finger_joint2",
    "multiplier": 1.0, "offset": 0.0}],
  "actuators": [..., # 8 actuator ports
    {"id": "actuator8", "transmission": {
      "finger_joint1": 0.5,
      "finger_joint2": 0.5}}}],
  "tool": {"body": "hand", "T_body_tool": ...},
  "redundancy_joint": "joint3", ...}

comp = compile_graph(physical) # q_f2 = q_f1
B = comp.cmg["actuation"]["moment_matrix"]
g = GeneratedTaskGraph(comp) # 6-D tool target
# lift E, velocity v = E u, curvature term c:
E, v, c, _ = curvature(g, g.lift(q), u)
ET = g.physical_map(E) # tree rows of E
tau = B @ f + Jw.T @ w # ports and wrench
Mr = ET.T @ MT @ ET # reduced inertia
ud = solve(Mr, ET.T @ (tau - hT - MT @ cT))
# virtual-power defect:
defect = tau @ vT - (ET.T @ tau) @ u

```

Listing 2. Abridged Panda input and reduced-dynamics calls. Here v_T and c_T are the tree rows of $v = Eu$ and of the curvature term, f collects the actuator efforts, and w is the tool wrench with Jacobian J_w ; the last line evaluates the virtual-power identity (12).

lift. Auxiliary gauge freedoms must be removed before an inertia inverse is used. Moving actuator bodies retain their own inertia, and supplied rotor or reflected-inertia models are counted once, consistently with the tree model. These expressions follow standard constraint embedding [10], [12]; RoboCompiler supplies the compatible maps E_T and L and the curvature term automatically from the physical graph. Listing 2 shows these calls for Panda. The equal finger displacements enter as the affine coupling $\gamma(q) = q_{f2} - q_{f1} = 0$, and the gripper motor is a single actuator port acting on both fingers.

V. INTRINSIC VELOCITY SELECTION AND RECONFIGURATION

A. A Physical Secondary Objective

At a regular task value, the task fiber is

$$\mathcal{F}_x = \{q \in \mathcal{C} : f_x(q) = x\}, \quad \dim \mathcal{F}_x = d - \text{rank } J. \quad (15)$$

Passive loop coordinates do not increase this dimension. Let $\Phi(q)$ be a differentiable physical preference, and define $g = E^\top N^\top D\Phi^\top$, so that $\dot{\Phi} = g^\top u$. For instance, with finite actuator stroke limits and $s_i = (\ell_i - \ell_i^-)/(\ell_i^+ - \ell_i^-)$, an interior preference is

$$\Phi(q) = -\sum_i \beta_i \log(s_i(1 - s_i)), \quad \beta_i > 0. \quad (16)$$

This objective penalizes proximity to the specified stroke limits.

Choose a positive physical motion metric whose reduced matrix is $W \succ 0$. One admissible choice is $W = M_r$ when its definiteness conditions hold. For task error $e_x = x - x_d$ in the chosen task chart, prescribe $w = \dot{x}_d - K_x e_x$, where

$K_x \succ 0$. In the compiled coordinates, the constrained velocity problem (Fig. 4) is

$$u^* = \arg \min_u \frac{1}{2} u^\top W u + k_\Phi g^\top u \quad (17)$$

subject to $Ju = w, \quad Au \leq b,$

with $k_\Phi > 0$ and physically specified bounds. For example, actuator speed bounds use $A = \text{col}(L, -L)$ and $b = \text{col}(\dot{\ell}^+, -\dot{\ell}^-)$. Other kinematic restrictions may be appended with their correct reduced maps. Feasibility and $W \succ 0$ imply a unique solution. An infeasible task is rejected explicitly or handled by a declared relaxation policy, so the task equality is never softened silently.

Without inequalities and with full row rank J , Eq. (17) reduces to

$$J_W^\# = W^{-1} J^\top (JW^{-1} J^\top)^{-1}, \quad (18)$$

$$u^* = J_W^\# w - k_\Phi (I - J_W^\# J) W^{-1} g.$$

This is weighted redundancy resolution [19]–[21], posed here directly in the compiled independent coordinates with the reduced physical metric W . Under exact feasible execution, $\dot{e}_x = -K_x e_x$. At a stationary task with $w = 0$ and no inequalities,

$$\dot{\Phi} = -k_\Phi g^\top (W^{-1} - W^{-1} J^\top (JW^{-1} J^\top)^{-1} JW^{-1}) g \leq 0. \quad (19)$$

The physical preference Φ is therefore non-increasing along the resulting self-motion. Floating-base or otherwise underactuated systems additionally require a dynamic/contact feasibility layer to realize this velocity with admissible actuator efforts and contact forces.

B. Consistency Under Representation Changes

Two representations are equivalent here when they describe the same physical configurations, body velocities, actuator ports, and task on an overlapping regular branch. Use $\mathcal{V} = Bu$ for the stacked physical body velocities in common frames. After removing auxiliary gauge freedoms, assume both representations give full-rank bases of the same tangent space. There is then an invertible $F(q)$ such that

$$\tilde{B} = BF, \quad u = F\tilde{u}. \quad (20)$$

The corresponding reduced quantities must satisfy

$$\tilde{J} = JF, \quad \tilde{L} = LF, \quad \tilde{A} = AF, \quad \tilde{W} = F^\top W F, \quad (21)$$

$$\tilde{g} = F^\top g, \quad \tilde{\tau}_r = F^\top \tau_r.$$

Task coordinates, the physical objective, and bounds are held fixed. Changing their physical meaning is not a reparameterization.

Proposition 1 (Motion and power consistency): Under the preceding equivalence assumptions, with a feasible problem (17) and $W \succ 0$, the transformed problem has a unique solution satisfying $u^* = F\tilde{u}^*$. Both representations yield the same body velocities, task velocity, actuator velocity, and mechanical power.

Proof. The bijection $u = F\tilde{u}$ maps the feasible sets onto each other by Eq. (21), and preserves both terms of the objective. Strict convexity gives the stated correspondence

of minimizers. Substitution into Bu , Ju , and Lu preserves physical velocities. Finally, $\tilde{u}^\top \tilde{\tau}_r = u^\top \tau_r = f^\top \dot{\ell}$.

The proposition carries coordinate covariance through all generated task, effort, and inequality interfaces and specifies the transformation contract (21). In particular, replacing a transformed physical metric by the identity in every chart generally changes the optimization problem.

For dynamics, configuration-dependent F also produces an acceleration term:

$$\tilde{M}_r = F^\top M_r F, \quad \tilde{h}_r = F^\top (h_r + M_r \dot{F} \tilde{u}). \quad (22)$$

Therefore, matching reduced mass matrices alone is insufficient to establish equivalent dynamic equations.

C. Three Meanings of Reconfiguration

Model reconfiguration changes dimensions, attachments, or joint/module data between runs. It is handled by recompiling the affected paths, closures, transmissions, and inertia mappings from the edited physical records; the recompiled model describes the changed mechanism.

Computational reconfiguration changes a tree, cycle basis, or valid local velocity partition while preserving the physical mechanism. On an overlap, state and speed must be transferred through their actual coordinate transformations, including $u = F\tilde{u}$. A new partition is accepted only after complete closure and rank conditions are re-established. The proposition applies to a valid switch.

Physical reconfiguration during motion adds or removes actual connections. Latching, grasping, impacts, and changing contact modes alter the admissible velocities. RoboCompiler compiles each support mode as a fixed-topology closure and schedules switches between compiled modes, as in the Go2 reference with 110 support transitions (Section VI-E); in the reported executions, impacts and contact transitions are resolved by the contact models of the executing backends. Nonholonomic rolling constraints act at the velocity level and can enter through a separate motion layer.

VI. EXPERIMENTAL EVALUATION

A. Robots, Environments, and Evaluation Protocol

We evaluate RoboCompiler on five simulated robots (Table III and Fig. 5). The Komatsu excavator, Kangaroo, and six-UPS Stewart platform contain physical closed chains. Go2 and Panda have physical trees augmented by foot-task and support constraints, or by a finger coupling and tool target. The excavator and Stewart compilers select trees and cuts from physical records; Kangaroo uses declared cuts. The systems exercise moving actuator bodies, coordinate partitions, floating bases, and task-dependent constraints.

MuJoCo and Isaac Sim/PhysX integrate native joints and loop constraints, together with unilateral contact where applicable. Pinocchio supplies independent rigid-body and constrained-dynamics calculations; its task executions use separately implemented contact or external-load models. Within each matched experiment, the physical parameters, references, and actuator definitions are held fixed. The task studies use MuJoCo 3.3.7, Pinocchio 3.8.0, and Isaac Sim 6.1; the separate

```

physical = { # Stewart records, abridged
  "bodies": [...], # 21, with mass and inertia
  "joints": [ # 25 undirected edges
    {"id": "leg_0_length", "type": "prismatic",
     "body_a": "leg_0_barrel",
     "body_b": "leg_0_rod", ...},
    {"type": "spherical", "body_a": "leg_0_rod",
     "body_b": "platform", ...}, # six such
    ...],
  "actuators": [...]} # six leg-force ports

for s in spherical_ids: # six tree-joint choices
  comp = compile_graph(
    physical, preferred_spherical_tree=s)
  # tree: 18 scalar + 3 chart coordinates;
  # 5 spherical chords: 30 residual rows
  graph = comp.graph(q0) # q0: closed seed
  E, info = PACDM(graph).mapping(graph.lift(q0))

```

Listing 3. Abridged Stewart input and compiler calls. The six platform joints are ordinary spherical records. For each choice, the compiler places that joint in the tree as three chart coordinates and closes the other five as chords; E is the lift in (9).

solver comparison in Table V also tests Pinocchio 4.1 and MuJoCo 3.14.0. The Kangaroo, Panda, and Go2 models are derived from public MuJoCo models [25], [26].

The evaluation tests mechanical consistency, computational savings, use of the actuation interface, and reuse in independent plants. Statewise ideal-support checks are separated from executions with changing native contact. We likewise distinguish physical endpoint gaps from augmented closure residuals.

B. Representation and Mechanical Consistency

Compilation under changed data and representations. For Stewart, twelve variants covering geometry, payload, inertia, body-frame expression, and record or actuator order are compiled under all six spherical tree-joint choices. All 72 models pass five feasible-configuration checks, giving 360 witnesses. The generated physical tree has 21 rather than 24 coordinates, while retaining six independent actuator lengths (Listing 3). Between paired representations of the same mechanism, the largest normalized reduced-inertia discrepancy is 3.77×10^{-15} and the largest task-map entry difference is 2.89×10^{-15} ; the maximum physical closure gap is 5.72×10^{-16} m. At perturbed off-manifold states, generated differentials agree with centered directional differences within 3.01×10^{-10} , and their observed nonzero entries remain inside the generated sparsity pattern.

The excavator compiler is checked on 120 closed configurations: twelve physical-data variants, two independent-coordinate partitions, and five configurations per pair. Both joint and cylinder-stroke partitions retain seven independent speeds. Across 48 dynamics states, their projected accelerations agree with independently assembled KKT and native Pinocchio solutions to 5.7×10^{-13} in the force-equivalent metric defined below; the two partitions agree to 1.94×10^{-15} . For Go2, twelve physical-data variants generate 144 task/support plans and 720 passing configuration checks. These checks compare valid representations of each edited mechanism, exercising recompilation from physical records.

Independent constrained dynamics. Let $a = \dot{v}_T$ denote generalized acceleration in the common tree-velocity con-

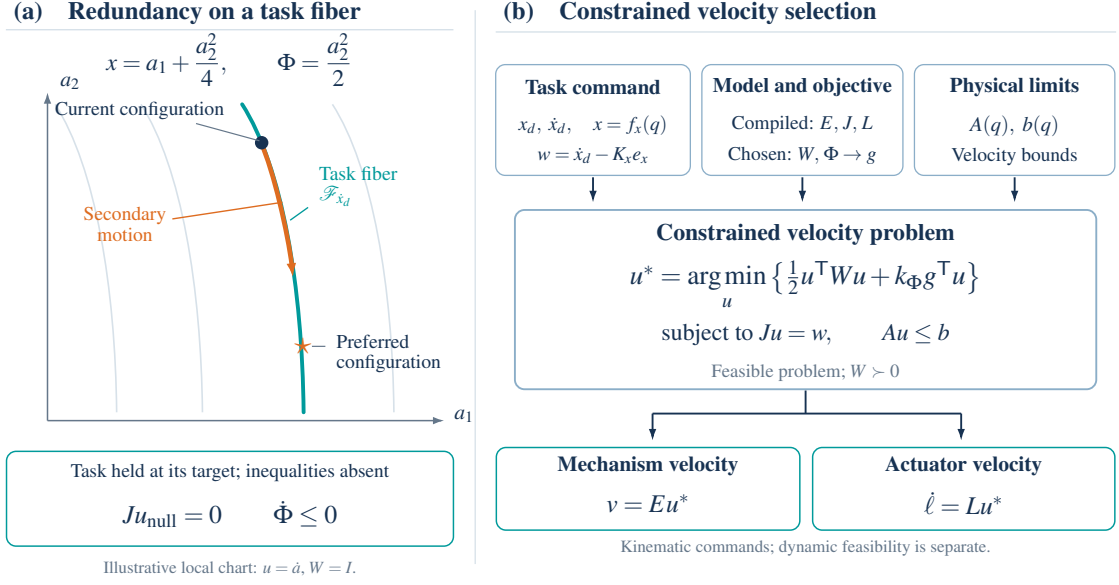


Fig. 4. Constrained velocity selection with the compiled interfaces. (a) On a task fiber, the secondary objective Φ generates self-motion toward a preferred configuration while the task is held ($\dot{\Phi} \leq 0$); illustrative local chart with $u = \dot{a}$ and $W = I$. (b) Task commands, the compiled maps E , J , and L , the chosen metric W and objective differential g , and physical velocity bounds define problem (17); its solution gives the mechanism velocity $v = Eu^*$ and actuator velocity $\dot{\ell} = Lu^*$.

TABLE III
EVALUATION SYSTEMS AND NOMINAL NATIVE ISAAC SIM/PHYSX EXECUTIONS. THE LAST COLUMN REPORTS TASK-SPECIFIC METRICS.

Robot	Mechanism or imposed constraints	Task	Native PhysX result
Komatsu PC138US-8	Nine revolute cuts; seven independent arm speeds	Excavation and settled soil transfer	92.153 kg delivered; 11.46- μ m peak arm gap
Unitree Go2	Tree; four foot targets and two-to four-foot support	26-s rail, turn, gate, and docking course	6.926-mm base-position RMS ($t \geq 2$ s)
Franka Panda	Tree; equal finger displacements and six-dimensional tool target	22-s grasp, inspection, and keyed-socket placement	0.133-mm final planar placement error
Kangaroo	Sixteen point and eight universal cuts; six base and twelve motor speeds	10-s landing, crouch, and push recovery	120.078-mm crouch; 0.629- μ m peak point gap
Six-UPS Stewart	Six limbs with spherical closure; six independent lengths	22-s six-axis inspection with platform wrenches	0.717-mm platform-position RMS; 6.454- μ m peak gap

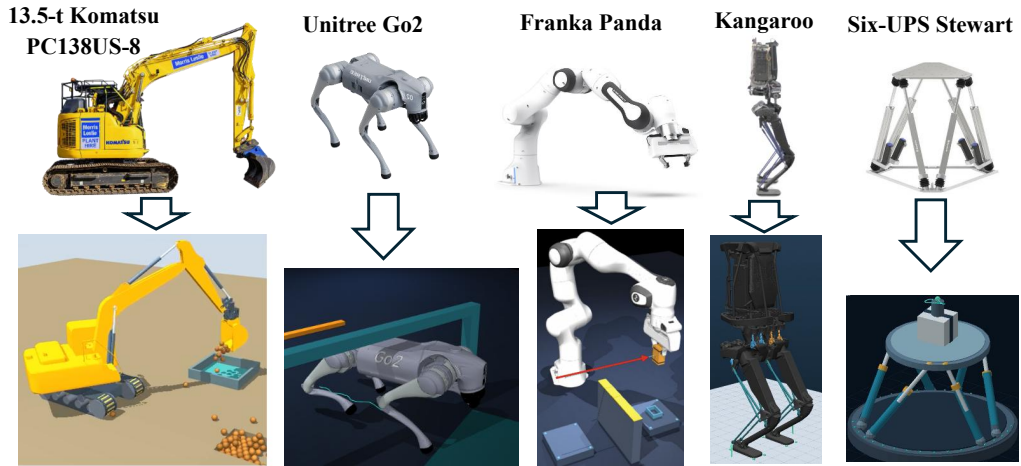


Fig. 5. The five evaluation platforms (top) and the simulation models used in all experiments (bottom): the Komatsu PC138US-8 excavator, Unitree Go2, Franka Panda, Kangaroo, and six-UPS Stewart platform.

vention. For the comparison in Table V, define $\bar{a} = a$ for Stewart and the excavator. For Kangaroo, $\bar{a}$ stacks the pelvis classical linear and angular accelerations in world axes and the 76 joint accelerations, avoiding dependence on the floating-base parameterization. Against the corresponding reference quantities, we report

$$\begin{aligned}\delta_a &= \frac{\|\bar{a} - \bar{a}_{\text{ref}}\|_\infty}{\max(1, \|\bar{a}_{\text{ref}}\|_\infty)}, \\ \delta_f &= \frac{\|M_T(a - a_{\text{ref}})\|_\infty}{\max(1, \|\tau_T\|_\infty, \|h_T\|_\infty)}.\end{aligned}\quad (23)$$

The force metric uses generalized accelerations and the reference inertia and bias terms. Coordinates and SI conventions are matched within each model. With physical constraint matrix $J_{c,T}$ and drift $\gamma_{c,T}$, the acceleration-level residual is $r_c = \|J_{c,T}a + \gamma_{c,T}\|_\infty$. Its components retain the units of their constraint rows, and all three metrics are compared within each robot.

Sampling and reference. Table V uses sets A and B. Set A contains 1101 recorded Stewart reference knots and 48 recorded dynamics witnesses each for the excavator and Kangaroo. Set B adds 300, 252, and 152 states, respectively, with configurations sampled from the same reference motions and randomized independent velocities, actuator efforts, and external wrenches. Independent velocities are mapped through $v_T = E_T(q)u$.

At each state, Pinocchio 4.1 evaluates M_T , h_T , $J_{c,T}$, and $\gamma_{c,T}$ in double precision. The reference uses a 50-significant-digit SVD null-space solve, retaining singular values above $10^{-9}\sigma_1$ and treating redundant rows in the least-squares sense at the selected numerical rank. Returned accelerations are rounded to double precision. Extended precision reduces solver round-off in the reference.

Every method receives the same physical configuration, generalized velocity, applied efforts, and support mode, converted to its native coordinate conventions. Table IV specifies the recorded and randomized states. Configurations are sampled from the 1101-knot Stewart reference (0–22 s at 20-ms intervals), the 4201-sample excavator route, and the 401-sample Kangaroo reference motion. Randomized states use a NumPy generator initialized with seed 20260927 for each robot, with a fixed draw order.

Independent velocities are mapped through $v_T = E_T(q)u$. The independent variables are the six actuator lengths for Stewart, the seven inputs of the excavator’s joint-space partition, and the coordinates selected by Kangaroo’s support plan. Kangaroo has 18, 12, or 6 independent speeds in floating, single-support, or double-support mode, respectively. Ideal sole welds are anchored at the current sole frames. External wrenches are expressed in world axes and applied at the platform origin, bucket frame, or pelvis origin for Stewart, the excavator, or Kangaroo, respectively.

The Stewart velocity interval covers the recorded peak actuator speed of approximately 0.23 m/s, while the sampled forces remain within the 900-N actuator limit. Together, sets A and B contain 1401 Stewart, 300 excavator, and 200 Kangaroo states.

Reference construction and numerical rank. Let $J_{c,T} = U\Sigma V^\top$, and let U_r , V_r , and Σ_r contain the retained singular vectors and values. With Z containing the remaining right singular vectors, the reference is computed as

$$\begin{aligned}a_p &= -V_r \Sigma_r^{-1} U_r^\top \gamma_{c,T}, \\ (Z^\top M_T Z)y &= Z^\top (\tau_T - h_T - M_T a_p), \\ a_{\text{ref}} &= a_p + Zy.\end{aligned}\quad (24)$$

The particular acceleration a_p uses a truncated-SVD least-squares treatment of the constraint equations. At the selected numerical rank, positive definiteness of $Z^\top M_T Z$ gives a unique projected solution.

The retained rank is 18 of 18 rows for Stewart and 16 of 54 rows for the excavator. For Kangaroo, it is 64 of 80 rows in floating motion, 70 of 86 rows with one welded sole, and 76 of 92 rows with both soles welded. These ranks are unchanged across the sampled states and agree with the double-precision rank checks.

Reference sensitivity. To distinguish solver sensitivity from the pelvis-based Kangaroo comparison in Table V, define the normalized tree-coordinate acceleration discrepancy

$$\delta_{\text{tree}} = \frac{\|a - a_{\text{ref}}\|_\infty}{\max(1, \|a_{\text{ref}}\|_\infty)}.\quad (25)$$

The largest differences between double- and extended-precision solves are 1.6×10^{-13} , 4.4×10^{-9} , and 1.2×10^{-11} in δ_{tree} for Stewart, the excavator, and Kangaroo, respectively. These values measure solver sensitivity with the model inputs held fixed.

For the excavator, alternative well-conditioned independent subsets of its redundant constraint rows change the reference by up to 6.5×10^{-10} in δ_a , with a median of 4.1×10^{-11} , and by up to 2.3×10^{-12} in δ_f . These observed spreads provide an empirical estimate of reference sensitivity, rather than a rigorous error bound. Differences below this estimated floor indicate agreement with the chosen reference without resolving an absolute accuracy ordering.

RoboCompiler’s maximum δ_a is 1.6×10^{-8} , 5.1×10^{-7} , and 4.4×10^{-12} for Stewart, the excavator, and Kangaroo. These are smaller than the tested MuJoCo 3.14.0 maxima by approximately 4.2×10^5 , 4.7×10^4 , and 6.1×10^{10} . Because MuJoCo uses regularized equalities, the ratios measure agreement with the hard-constraint reference under the reported settings. Against Pinocchio’s `lcaba` with $\mu = 10^{-6}$ (with the Schur recipe on Kangaroo), RoboCompiler attains lower median δ_a and maximum δ_f on all three mechanisms and lower maximum δ_a on the excavator and Kangaroo; on Kangaroo, it also improves on `lcaba` with $\mu = 10^{-2}$ in every accuracy metric. Pinocchio’s `constraintDynamics`, with the stated Kangaroo Schur recipe, attains the smallest discrepancies overall.

For the excavator, RoboCompiler’s maximum $\delta_f = 1.1 \times 10^{-12}$ lies below the estimated 2.3×10^{-12} reference floor arising from alternate treatments of redundant rows, as do the δ_f entries of `lcaba` with $\mu = 10^{-2}$ and `constraintDynamics`; at the reference resolution, these three solvers are indistinguishable in δ_f . Call times compare RoboCompiler’s Python reference implementation with

TABLE IV

SAMPLING PROTOCOL FOR THE DYNAMICS COMPARISON. $\mathcal{U}(\pm a)$ DENOTES INDEPENDENT UNIFORM DRAWS IN $[-a, a]$ FOR EACH COMPONENT. SET A CONTAINS RECORDED STATES; SET B CONTAINS RANDOMIZED VELOCITIES, EFFORTS, AND WRENCHES AT CONFIGURATIONS SAMPLED FROM THE REFERENCE MOTIONS.

Robot	Set A: recorded states	Set B: randomized states
Stewart	All 1101 reference knots, with recorded actuator velocities and feedforward forces; no external wrench.	300 states. Reference knot sampled uniformly; actuator velocities $\mathcal{U}(\pm 0.25)$ m/s; actuator forces $\mathcal{U}(\pm 400)$ N; external force components $\mathcal{U}(\pm 120)$ N and moment components $\mathcal{U}(\pm 12)$ N m.
Excavator	48 dynamics witnesses at route indices evenly spaced from 20 to 4181.	252 states. Route index sampled uniformly from 20–4180; independent velocities $\mathcal{U}(\pm 0.3)$ rad/s. Each effort is sampled as $\mathcal{U}(\pm 1)$ times its scale: 5, 5, 30, 150, 150, and 500 kN for the six cylinders, and 1 and 25 kN m for the rotary drives. External force components are $\mathcal{U}(\pm 5)$ kN and moment components are $\mathcal{U}(\pm 2)$ kN m.
Kangaroo	48 dynamics witnesses at motion indices evenly spaced from 8 to 392, cycling through floating, left-sole, right-sole, and double-support modes.	152 states, with 38 per support mode. Motion index sampled uniformly from 8–392; independent velocities $\mathcal{U}(\pm 0.12)$ in their respective m/s or rad/s units; motor efforts $\mathcal{U}(\pm 200)$ N. External force components use ranges ± 15 , ± 15 , and ± 10 N; moment components use $\mathcal{U}(\pm 2)$ N m.

compiled-library calls on precomputed arrays (except for the complete Kangaroo Schur recipe) and therefore reflect implementation-specific costs with different scopes; the computational benefit of compilation is measured within a common implementation in Section VI-C.

Floating bases, support modes, and port power. Separate double-precision checks evaluate 48 Go2 states spanning all eleven tested two-, three-, and four-foot ideal-support sets. Their constraint ranks are 6, 9, and 12, leaving 12, 9, and 6 independent speeds. Maximum normalized acceleration discrepancies are 3.225×10^{-13} against native Pinocchio and 3.252×10^{-13} against independently assembled KKT. For Kangaroo, 48 states cover free motion, either sole welded, and both soles welded. Ideal sole constraints increase the rank from 64 to 70 or 76, leaving 18, 12, or 6 independent speeds; the corresponding discrepancy maxima are 1.86×10^{-12} and 1.85×10^{-12} . Kangaroo’s maximum actuator virtual-power defect is 1.43×10^{-14} W. Panda’s sixteen mechanics configurations give a maximum port virtual-work defect of 4.26×10^{-14} W. These checks test the tangent, acceleration, and dual-effort interfaces at specified support modes; contact transitions are exercised in the native executions of Section VI-E.

Acceleration curvature. Figure 6 connects independent dynamics checks to an omission study. For the 21-coordinate compiled Stewart tree, 48 feasible states give a maximum normalized acceleration discrepancy of 3.051×10^{-13} against native Pinocchio. At four times the prescribed actuator speed, the maximum physical point-acceleration residual is 8.876×10^{-11} m/s² with curvature and 1.010 m/s² without it. Go2 gives corresponding residuals of 8.676×10^{-11} and 1.445 m/s². The curvature correction thus lowers the acceleration residual by about ten orders of magnitude, showing that acceleration compatibility requires it in addition to position closure and a valid tangent map. The curvature is evaluated by a centered directional difference of the analytic closure Jacobian (Listing 4).

C. Generated Evaluation and Selective Updates

The computational studies distinguish three effects: replacing global path evaluation by generated cycle-local evaluation,

```
def jdot_qdot(C, q, qdot, h0=1e-5):
    """Cdot(q, qdot) qdot from the analytic
    closure Jacobian C(q), by a centered
    directional difference along qdot."""
    h = h0 / max(1.0, np.max(np.abs(qdot)))
    dC = (C(q + h*qdot) - C(q - h*qdot)) / (2*h)
    return dC @ qdot # second-order accurate

v = E @ u
cv = S @ jdot_qdot(C, q, v) # selected rows
c = np.zeros_like(v) # c = Edot u
c[p] = -solve(S @ C(q)[:, p], cv)
```

Listing 4. Velocity-product and curvature terms. C returns the analytic closure Jacobian C' of (5) in chart coordinates, where $v = \dot{q}$; no coordinate moves by more than h_0 . With S and p from (8), the last lines form $c = \dot{E}u$, whose tree rows enter the bias term of (14). Some workflows normalize by $\|\dot{q}\|_2$ or use $h_0 = 10^{-6}$, which keeps this bound.

reusing modules whose complete inputs are unchanged, and supplying analytic rather than numerical corrector derivatives. Table VI summarizes matched comparisons.

For fixed excavator cuts, the generated residual and analytic Jacobian agree with the original evaluator within 7.44×10^{-15} . Dependency grouping reduces the largest correction block from 25 to ten coordinates (Listing 5). Over a 42-s, 4201-sample reference, loop-free exclusion and modular updates reduce summed correction-plus-output-map time by 25.3% in joint coordinates and 32.1% with held cylinder strokes. The seven independent speeds are preserved, and the reduced dynamics remain physically coupled.

Go2’s local edits hold the prescribed base pose fixed, so changing one foot target requires correction of only its three-coordinate leg block (Listing 6). One- and two-foot updates reduce median update time, including output mapping, by 61.8% and 28.1%, respectively, relative to generated global updates. For Kangaroo, common-ancestor pruning limits closure paths to at most three joint steps. The table separately measures the global evaluator and one-module update, including output-map construction. These comparisons preserve the PACDM equations while changing evaluation or scheduling.

Stewart separates representation, prediction, and derivative effects (Fig. 7).

Over three traversals of 1101 knots, all four tested con-

TABLE V

CONSTRAINED FORWARD DYNAMICS AT IDENTICAL FEASIBLE STATES. ERRORS USE A 50-DIGIT HARD-CONSTRAINT KKT REFERENCE AND EQ. (23); $r_c = \|J_{c,T}a + \gamma_{c,T}\|_\infty$ HAS COMPONENTS IN m/s^2 , rad/s^2 , OR s^{-2} FOR UNIVERSAL-AXIS ROWS. TIME t , THE MEDIAN OF PER-STATE MEAN CALL TIMES, IS MEASURED SEPARATELY ON 100 STEWART/EXCAVATOR OR 40 KANGAROO STATES, ON ONE 2.80-GHz XEON CORE WITH ONE BLAS THREAD. ROBOCOMPILER IS A PYTHON REFERENCE IMPLEMENTATION. PINOCCHIO TIMINGS COVER THE NATIVE SOLVER CALL; MUJOCo TIMINGS INCLUDE NATIVE-ARRAY ASSIGNMENT AND `MJ_FORWARD`; KANGAROO TIMINGS COVER THE COMPLETE SCHUR RECIPE. STATE CONVERSION AND MODEL CONSTRUCTION ARE EXCLUDED FROM THE NATIVE TIMING SCOPES. [†]PINOCCHIO 4.1 LACKS NATIVE UNIVERSAL-LOOP ROWS; EIGHT ROWS ARE ADDED BY A SCHUR RECIPE REQUIRING TEN NATIVE SOLVER CALLS PER ACCELERATION. [‡]BELOW THE ESTIMATED EXCAVATOR REFERENCE FLOOR FROM ALTERNATE WELL-CONDITIONED INDEPENDENT SUBSETS OF REDUNDANT ROWS: δ_a MAX 6.5×10^{-10} , MEDIAN 4.1×10^{-11} ; δ_f MAX 2.3×10^{-12} . SUCH ENTRIES AGREE WITH THE REFERENCE TO WITHIN ITS RESOLUTION AND ARE MUTUALLY INDISTINGUISHABLE.

Method	δ_a max	δ_a median	δ_f max	r_c max	t
<i>Stewart platform: 24 tree coordinates, 6 point cuts (18 rows, rank 18); 1,401 states</i>					
RoboCompiler (authors' code)	1.6×10^{-8}	2.5×10^{-10}	4.9×10^{-10}	2.4×10^{-7}	22.3 ms
Pinocchio 4.1 lcaba, $\mu=10^{-6}$	9.3×10^{-9}	2.1×10^{-9}	1.5×10^{-9}	1.1×10^{-13}	19 μs
Pinocchio 4.1 lcaba, $\mu=10^{-2}$	9.8×10^{-13}	5.5×10^{-13}	1.4×10^{-13}	8.0×10^{-13}	28 μs
Pinocchio 4.1 constraintDynamics	7.6×10^{-14}	3.3×10^{-14}	3.1×10^{-15}	6.4×10^{-13}	16 μs
MuJoCo 3.3.7 equalities	2.6×10^{-1}	4.0×10^{-3}	3.0×10^{-3}	6.6×10^{-1}	22 μs
MuJoCo 3.14.0 equalities	6.7×10^{-3}	4.1×10^{-3}	1.5×10^{-4}	3.8×10^{-2}	20 μs
<i>Excavator arm: 23 tree coordinates, 9 revolute cuts as 18 point pairs (54 rows, rank 16); 300 states</i>					
RoboCompiler (authors' code)	5.1×10^{-7}	8.5×10^{-11}	$1.1 \times 10^{-12\dagger}$	3.4×10^{-11}	12.6 ms
Pinocchio 4.1 lcaba, $\mu=10^{-6}$	6.9×10^{-6}	6.9×10^{-7}	2.2×10^{-10}	3.1×10^{-12}	35 μs
Pinocchio 4.1 lcaba, $\mu=10^{-2}$	4.6×10^{-9}	1.6×10^{-10}	$2.8 \times 10^{-14\dagger}$	3.1×10^{-12}	109 μs
Pinocchio 4.1 constraintDynamics	$4.2 \times 10^{-12\dagger}$	$4.0 \times 10^{-13\dagger}$	$4.3 \times 10^{-15\dagger}$	7.2×10^{-12}	73 μs
MuJoCo 3.3.7 equalities	2.4×10^{-2}	2.3×10^{-3}	5.3×10^{-5}	3.2×10^{-1}	33 μs
MuJoCo 3.14.0 equalities	2.4×10^{-2}	2.3×10^{-3}	2.9×10^{-5}	2.7×10^{-1}	32 μs
<i>Kangaroo: 82 generalized velocities incl. base, 16 point + 8 universal cuts (80 rows, rank 64) + 6 per welded sole; 200 states</i>					
RoboCompiler (authors' code)	4.4×10^{-12}	1.4×10^{-13}	2.8×10^{-13}	8.0×10^{-10}	52.6 ms
lcaba + Schur [†] , $\mu=10^{-6}$	3.3×10^{-4}	5.1×10^{-5}	3.4×10^{-5}	5.1×10^{-3}	3.0 ms
lcaba + Schur [†] , $\mu=10^{-2}$	5.3×10^{-9}	1.2×10^{-9}	9.6×10^{-12}	2.1×10^{-5}	3.7 ms
constraintDynamics + Schur [†]	5.3×10^{-13}	3.9×10^{-15}	2.4×10^{-15}	1.4×10^{-11}	5.6 ms
MuJoCo 3.3.7 equalities	2.7×10^{-1}	2.4×10^{-3}	2.9×10^{-3}	1.5×10^1	125 μs
MuJoCo 3.14.0 equalities	2.7×10^{-1}	3.2×10^{-3}	2.9×10^{-3}	2.3×10^1	125 μs

TABLE VI

MATCHED COMPUTATIONAL ABLATIONS. TIMES ARE MEDIANS EXCEPT THE FINAL ROW, WHICH AVERAGES TWO COUNTERBALANCED RUNS PER EVALUATOR. SCHEDULED EXCAVATOR TIMES ARE MEDIAN SUMMED CORRECTION-PLUS-MAP TIMES OVER THREE TRAVERSALS. NUMERICAL DERIVATIVE PAIRS USE MATCHED CORRECTORS AND STOPPING CRITERIA. EACH ROW IS A MATCHED PAIR MEASURED ON A SINGLE HOST AND WORKLOAD. FD DENOTES FINITE DIFFERENCES.

Robot	Paired operation	Reference	Generated/local	Reduction
Excavator	Residual + analytic Jacobian (ms)	4.097	0.984	76.0%
Excavator	Joint-reference correction + maps (s)	24.117	18.026	25.3%
Excavator	Held-cylinder correction + maps (s)	24.493	16.628	32.1%
Go2	One-foot update (ms)	3.467	1.326	61.8%
Go2	Colored FD / analytic correction (ms)	9.657	4.311	55.4%
Kangaroo	Residual + analytic Jacobian (ms)	10.172	0.339	96.7%
Kangaroo	One-module update (ms)	8.963	1.979	77.9%
Stewart	Original / compiled assembly + maps (ms)	24.924	21.349	14.3%
Stewart	Dense FD / analytic correction (ms)	224.085	16.705	92.5%
Stewart	Colored FD / analytic correction (ms)	102.621	25.338	75.3%
Kangaroo	Matched 10-s Pinocchio rollout (s)	585.594	194.366	66.8%

tinuation methods accept all 3303 samples. Compiling the physical-tree representation reduces median assembly-plus-mapping time by 14.3%; tangent prediction reduces it by 33.2% relative to the 31.943-ms predictor-disabled case. In separate matched corrector trials, all 144 attempts per mode succeed. Analytic Jacobians reduce median fresh evaluations from 183 to three against dense three-point differences and from 63 to three against sparsity-colored differences. Dense pairs share the exact dense TRF subsolver; sparse pairs share LSMR. The benefit therefore persists when numerical differ-

entiation exploits the generated sparsity. Go2 similarly reduces fresh evaluations from 21 colored evaluations to three analytic evaluations.

The complete Kangaroo rollout shows that evaluator savings persist inside closed-loop integration (Fig. 8). Only the graph evaluator changes; the PACDM solver, dynamics, contact algorithm, controller, and 1-ms integration step remain identical. Counterbalanced original-generated-generated-original runs on one 2.10-GHz Xeon host with one BLAS thread reduce mean elapsed time from 585.594 to 194.366 s. Both versions

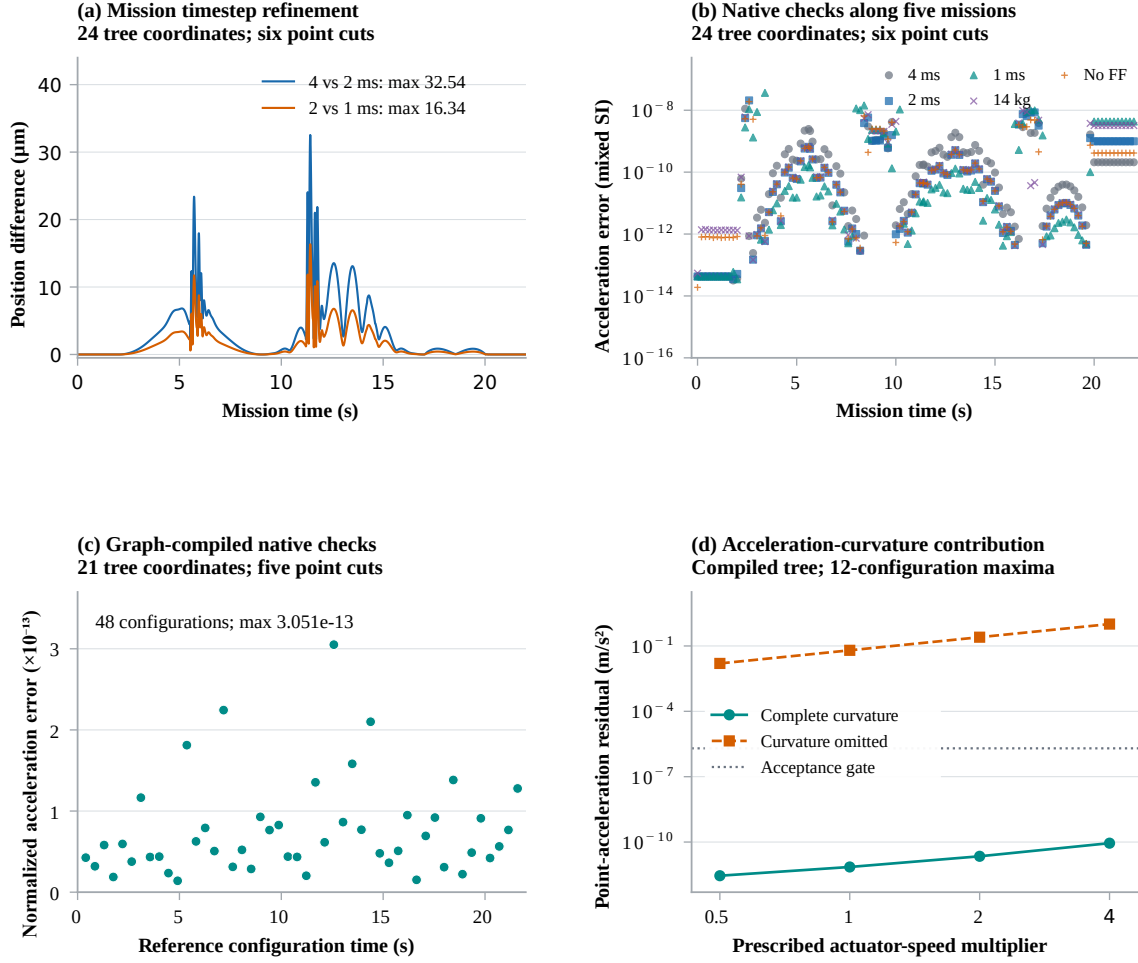


Fig. 6. Stewart dynamics verification. (a) Position differences under successive integration-step refinements. (b) Native constrained-acceleration checks at 113 states per mission case, in mixed SI coordinates. (c) Normalized acceleration discrepancies at 48 configurations of the compiled tree. (d) Physical point-acceleration residuals at four prescribed speed multipliers, with and without curvature; the dotted line denotes the recorded acceptance threshold. Panels (a,b) use the 24-coordinate mission tree with six point cuts; (c,d) use the 21-coordinate compiled tree with five cuts.

```

physical = { # excavator records, abridged
  "bodies": [...], # 25, with mass and inertia
  "joints": [
    # 34 edges; no cut is marked
    {"id": "q5", "type": "revolute", ...,
     "body_a": "body_54", "body_b": "body_3"},
    ...],
  "actuators": [...], # 8 actuator ports
  "partition_requests": {
    "joint_space": ["q23", "q7", "q4", "q0",
                   "q1", "q21", "q22"],
    "cylinder_space": ["q23", "p3", "p5", "p2",
                      "p1", "q21", "q22"]},
  "seed": {...}, "branch_window": {...}}

comp = compile_graph(physical, "joint_space")
comp.plan["cut_joint_ids"] # 9 revolute cuts
comp.plan["mobility"] # 7
comp.plan["module_sizes"] # [3, 3, 10, 3, 3, 3]
comp = compile_graph(physical, "cylinder_space")
comp.plan["module_sizes"] # [6, 10, 6, 3]

```

Listing 5. Abridged excavator input and compiler calls. No joint is marked as a cut: the compiler selects nine revolute cuts, including q5, checks each requested partition against the mobility, and groups the 25 dependent coordinates into modules, which are then corrected as in Listing 1.

```

physical = { # Go2 records, abridged
  "floating_root": "base",
  "bodies": [...], # 13, with mass and inertia
  "joints": [...], # 12 revolute edges
  "actuators": [...], # 12 motor ports
  "point_sites": [ # feet as task/contact sites
    {"id": "FL", "body": "FL_calf",
     "point_m": [-0.002, 0.0, -0.213], ...}, ...],
  "seed": {...}}

comp = compile_graph(physical)
# generated: base chart + 12 joints, and one
# three-coordinate module per foot site
plans = [comp.support_plan(s)
          for s in support_sets()] # 11 modes
solver = ModularSolver(comp, q0)
q, E, info = solver.step(np.r_[q_b, p_feet])
# q_b: base pose (6); p_feet: foot targets (12)

```

Listing 6. Abridged Go2 input and task-module calls. Feet are point sites, not closure joints. The compiler generates the floating-base chart, one three-coordinate module per foot, and the eleven two- to four-foot support plans; step re-solves only modules whose base or foot inputs changed.

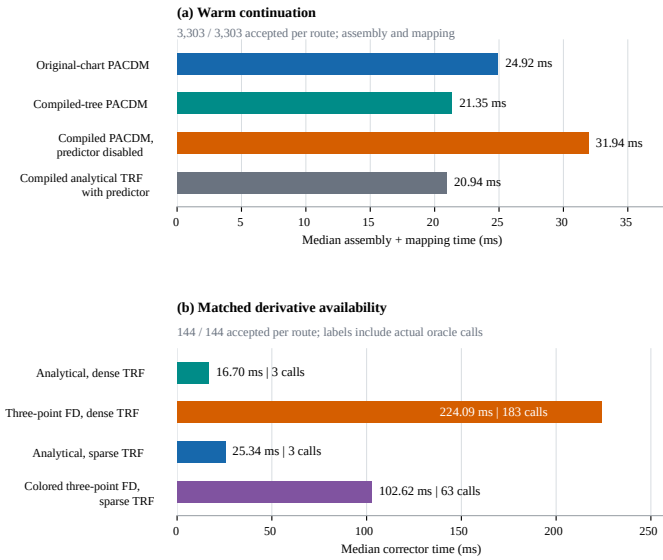


Fig. 7. Computational evaluation from the recorded full-profile run; bars show medians. (a) Assembly-plus-mapping time over three 1,101-knot traversals for the original and compiled PACDM representations, the compiled predictor-disabled route, and same-chart analytical TRF. (b) Matched TRF corrector timings with analytical or three-point finite-difference (FD) derivatives: dense modes share the exact dense subsolver; sparse/colored modes share LSMR. All attempts are accepted. Oracle counts include finite-difference perturbation evaluations. The timing scopes in the two panels are evaluated separately.

execute 10000 steps, 37870 fresh graph evaluations, and 111 correction iterations. Maximum differences are 7.64×10^{-14} m in motor position, 1.93×10^{-10} in reduced-velocity components, and 1.20×10^{-10} N s in net foot-impulse components. Because only the graph evaluator changes, the 3.0-fold speed-up is attributable to compilation alone, and the differences above confirm that the computed response is preserved.

D. Use of the Compiled Interfaces in Actuation

Matched feedforward ablations quantify the benefit of actuator commands derived from the compatible motion and dynamics interfaces. In Kangaroo’s 4-s fixed-pelvis experiment, both runs use the same reference, proportional gain 4×10^4 N/m, derivative gain 600 N s/m, and 100- μ s physics step. Adding reduced-dynamics feedforward decreases aggregate twelve-motor position RMS from 1.653 mm to 0.647 μ m. Every motor improves, and the peak actuator force decreases from 250.70 to 239.84 N (Fig. 9). This experiment isolates feedforward use under a fixed base; floating-base contact execution is evaluated separately below.

Feedforward derived from the same interface also reduces tracking error on Stewart and Panda. For Stewart, feedforward reduces nominal platform-position RMS from 2.375 to 0.717 mm in MuJoCo and direct PACDM–Pinocchio, and from 2.374 to 0.717 mm in PhysX. Increasing the actual payload from 8 to 14 kg while retaining the nominal feedforward model gives 0.960, 0.959, and 0.959 mm, respectively. All four PhysX cases, including timestep refinement, respect the stroke and force limits, with peak force below 152 N against the 900-N bound.

TABLE VII
Go2 NATIVE PHYSX PERTURBATIONS. RMS USES $t \geq 2$ S; FINAL ERROR IS AT 26 S. MOTOR UTILIZATION IS THE MAXIMUM COMMAND NORMALIZED BY THE AUTHORED LIMIT.

Case	RMS (mm)	Final (mm)	Motor (%)
Nominal, 1-ms step	6.926	0.179	68.43
0.5-ms step	6.736	0.474	64.22
Friction 0.55	6.855	0.265	72.26
1.5-kg payload	8.897	6.655	59.88
25% stronger pushes	7.237	0.222	100.00

For Panda, the same-reference MuJoCo arm ablation reduces tool-position RMS from 6.224 to 0.152 mm; the removed terms include inverse dynamics, armature-acceleration compensation, and damping compensation. A separate Pinocchio tool-wrench execution gives 6.326 versus 0.150 mm. Across Kangaroo, Stewart, and Panda, these ablations show that the generated motion-to-effort interface directly supports model-based feedforward; computational savings are isolated in Section VI-C.

E. Native Task Execution and Perturbations

Excavation with physical material transfer. The excavator completes filled-bed and exposed-face MuJoCo missions with 117.286 and 83.776 kg of identity-qualified settled delivery. A particle contributes only after it has been lifted in the bucket, released, and observed to settle inside the receiving region. Empty-bed and disabled-bucket-contact controls produce zero lifted or delivered mass. All 8856 PACDM updates across the two positive missions succeed without reacquisition, and the native arm gaps remain below 4.92 μ m.

The native PhysX execution completes the eleven phases in 42.001 s, with peak arm and track gaps of 11.46 μ m and 0.423 mm, respectively (Fig. 10). Its 4202 controller evaluations require no reacquisition. Eight fixed-base Pinocchio cutting-and-payload cases also complete all phases, with maximum acceleration-closure component 1.90×10^{-11} m/s². Those cases use prescribed smooth cutting and payload loads and validate force-driven mechanism execution.

Floating-base course execution. Go2 completes its 26-s course in MuJoCo, the separately integrated Pinocchio contact model, and PhysX. Nominal base-position RMS errors for $t \geq 2$ s are 10.60, 5.991, and 6.926 mm, respectively. The 2601-sample PACDM reference contains 110 scheduled support transitions and requires no fallback. During native PhysX execution, measured unilateral contacts produce two-, three-, and four-foot support. Table VII reports refinement, friction, payload, and push cases. All cases complete the course, including the 25%-stronger-push case, whose peak motor command reaches the authored limit. With an unmodeled 1.5-kg payload, Go2 still completes the course with 8.897-mm RMS and 6.655-mm final error.

Manipulation with coupled fingers. Panda completes the 22-s task with a 0.715-mm nominal final planar error in MuJoCo and the PhysX result in Table III. Four PhysX cases include a halved step, a 0.30-kg lower-friction cartridge, a 4-mm pickup

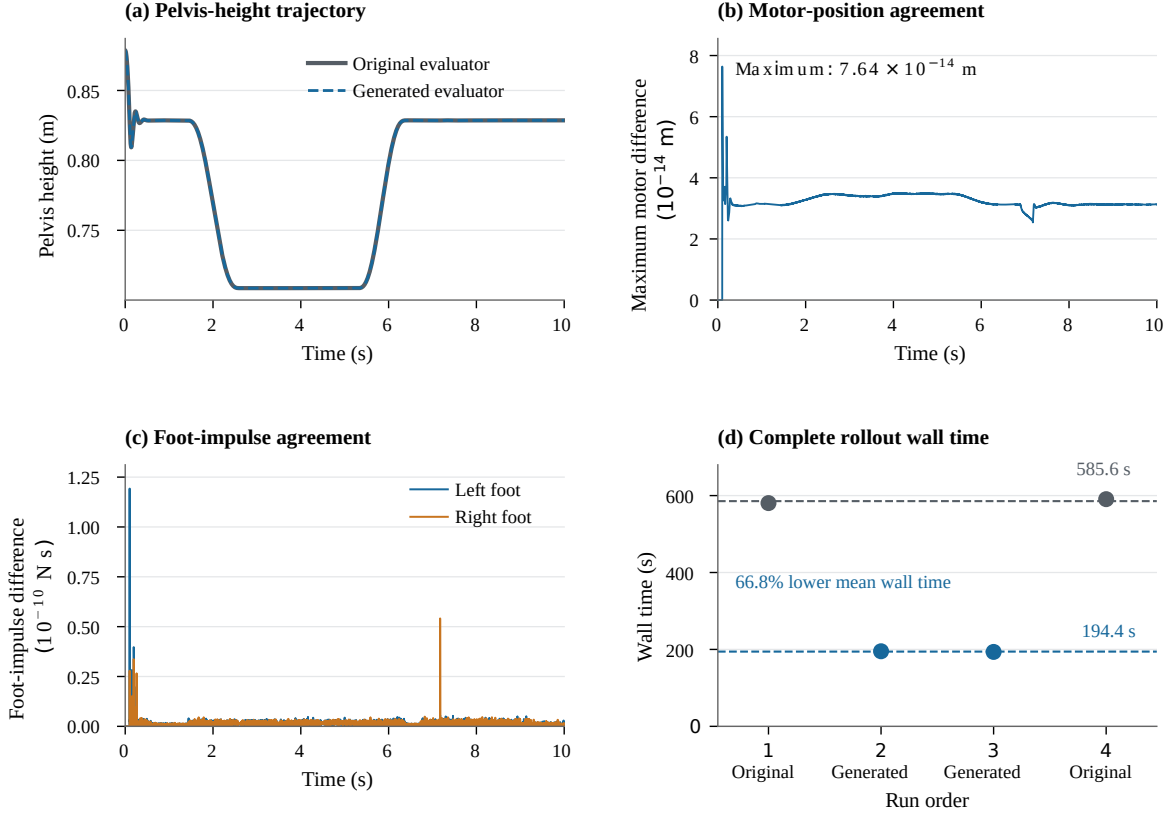


Fig. 8. Matched 10-s Kangaroo Pinocchio executions. (a) Pelvis height. (b) Motor-position differences. (c) Net per-foot impulse differences after summing the four contact-corner impulses. (d) Elapsed times and evaluator means from four counterbalanced runs. Histories use all 10001 samples; only the graph evaluator changes between implementations.

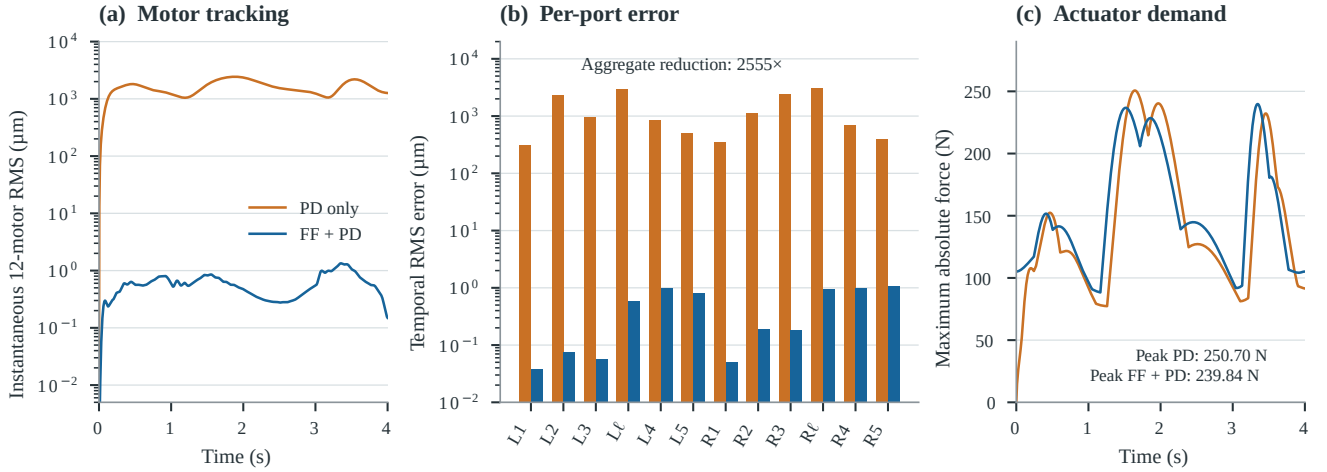


Fig. 9. Matched Kangaroo actuation experiment with a fixed pelvis. (a) Instantaneous RMS motor-position error. (b) Temporal RMS for each motor; $L\ell$ and $R\ell$ denote leg-length ports. (c) Maximum absolute actuator force. The reference, feedback gains, and physics step are identical; the changed component is the compatible reduced-dynamics feedforward.

offset, and a 1-mm socket side clearance. All maintain two-finger contact during the 8–16-s transfer and finish without arm saturation, command clipping, or unexpected-contact steps. The largest final planar error is 0.840 mm, and minimum payload-corner clearance during obstacle transfer is 92.541 mm. These results exercise the finger equality, tool reference, and effort interface together with native grasp contact; the

independent Pinocchio experiment instead applies a prescribed tool wrench.

Closed-chain landing and recovery. Kangaroo completes a 10-s sequence containing a 5-cm landing, 12-cm crouch, lateral and yaw motion, and a torso push. Across five shared MuJoCo/Pinocchio conditions, including lower friction, increased release height and push, timestep refinement, and

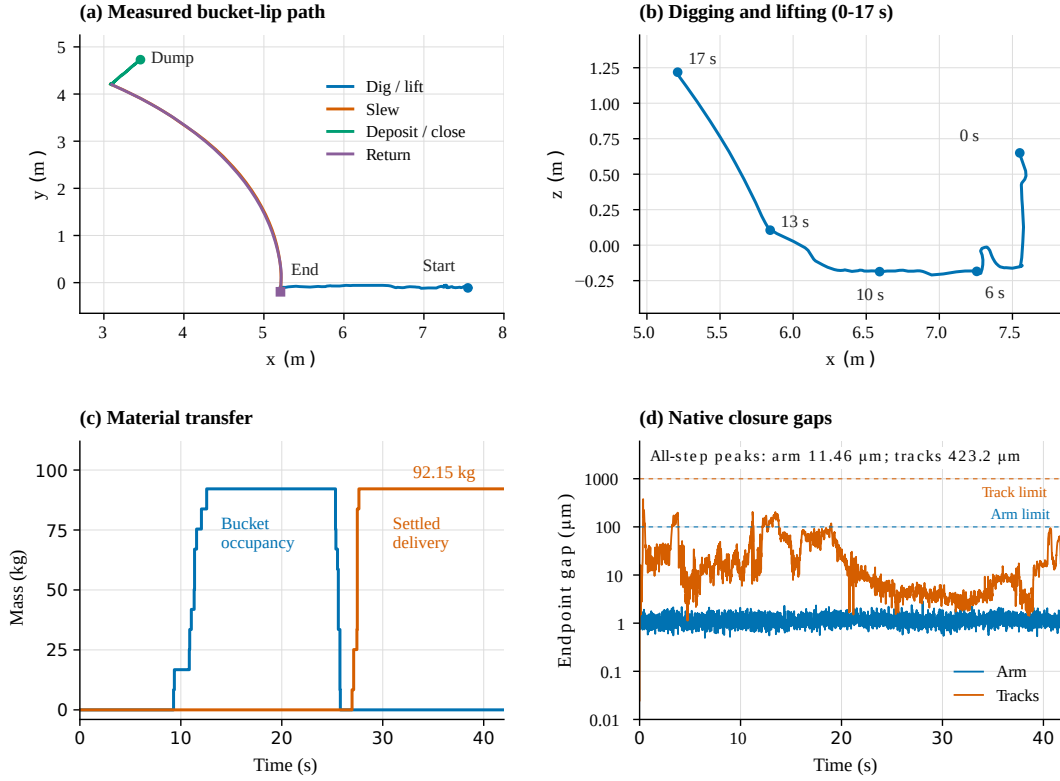


Fig. 10. Native PhysX excavation. (a) Complete bucket-lip plan-view trajectory. (b) Vertical trajectory during excavation and lifting (0–17 s). (c) Material retained in the bucket and identity-qualified settled delivery. (d) Physical arm and track closure gaps. Native plant gaps are measured separately from the controller’s assembled PACDM closure residual.

actuator response delay, the largest paired pelvis-position difference is 1.576 mm. Native PhysX also completes the task, achieving a 120.078-mm crouch within the individual force bounds. Its nominal maximum point gap is $0.629 \mu\text{m}$ and the dimensionless universal-axis residual is 3.05×10^{-4} . Reporting both tests the translational and angular parts of the universal-cut realization. MuJoCo and PhysX use a nominal $25\text{-}\mu\text{s}$ step, whereas the Pinocchio contact integration uses 1 ms; the agreement between MuJoCo and Pinocchio to within 1.576 mm therefore demonstrates model reuse across simulators whose integration steps differ by a factor of 40.

F. Independent Native-State and Work Checks

Trajectory agreement is complemented by reconstruction and mechanics checks at measured native states. For Go2, 55 PhysX readback checks verify the transferred masses, centers of mass, inertia tensors, and armatures. Twelve free-flight probes cover six states at both 0.1- and 0.05-ms steps. Measured acceleration agrees with the CMG/Pinocchio prediction to a maximum normalized 2-norm discrepancy of 1.25×10^{-5} ; this uses $\|a - a_{\text{ref}}\|_2 / \max(1, \|a_{\text{ref}}\|_2)$, distinct from Eq. (23). During the nominal course, native link poses and CMG forward kinematics give foot positions within $0.783 \mu\text{m}$.

Stewart provides a direct test of the closed-chain transfer (Fig. 11). Four PhysX missions comprise nominal, refined-step, increased-payload, and feedforward-disabled cases. At 46 prescribed timestamps per case, measured actuator lengths

and native configuration seeds are used to assemble PACDM afresh. All 184 assemblies retain full/passive augmented rank 36 and positive-definite reduced inertia. Reconstructed body origins agree with native states within $4.971 \mu\text{m}$; maximum center-of-mass linear and angular velocity discrepancies are $5.607 \times 10^{-5} \text{ m/s}$ and $3.582 \times 10^{-4} \text{ rad/s}$. These checks use the 24-coordinate mission representation (augmented rank 36) rather than the 21-coordinate compiled tree (augmented rank 30) of Section VI-B.

An independently evaluated Newton–Euler body sum supplies the reduced inertia and bias, including curvature. Since independent coordinates are actuator lengths in this experiment, its sampled force-balance residual is

$$r_{f,k} = M_{r,k} \frac{\dot{\ell}_{k+1} - \dot{\ell}_k}{\Delta t} + h_{r,k} - f_k - E_{T,k}^\top J_{w,k}^\top w_k, \quad (26)$$

where J_w maps tree velocities to platform twist and w_k is the applied platform wrench. The forward velocity difference is paired with the force held over that step. The largest per-actuator RMS residual across the four sampled missions is 2.612 mN. Unlike reference-trajectory comparison alone, this check connects the compiled inertial data, measured motion, curvature, and effort projection to the native plant response.

Numerical refinement provides a further check on integrated motion and mechanical work. For Stewart, consecutive 4/2-ms and 2/1-ms direct PACDM–Pinocchio runs reduce the maximum paired position difference from 32.541 to $16.337 \mu\text{m}$.

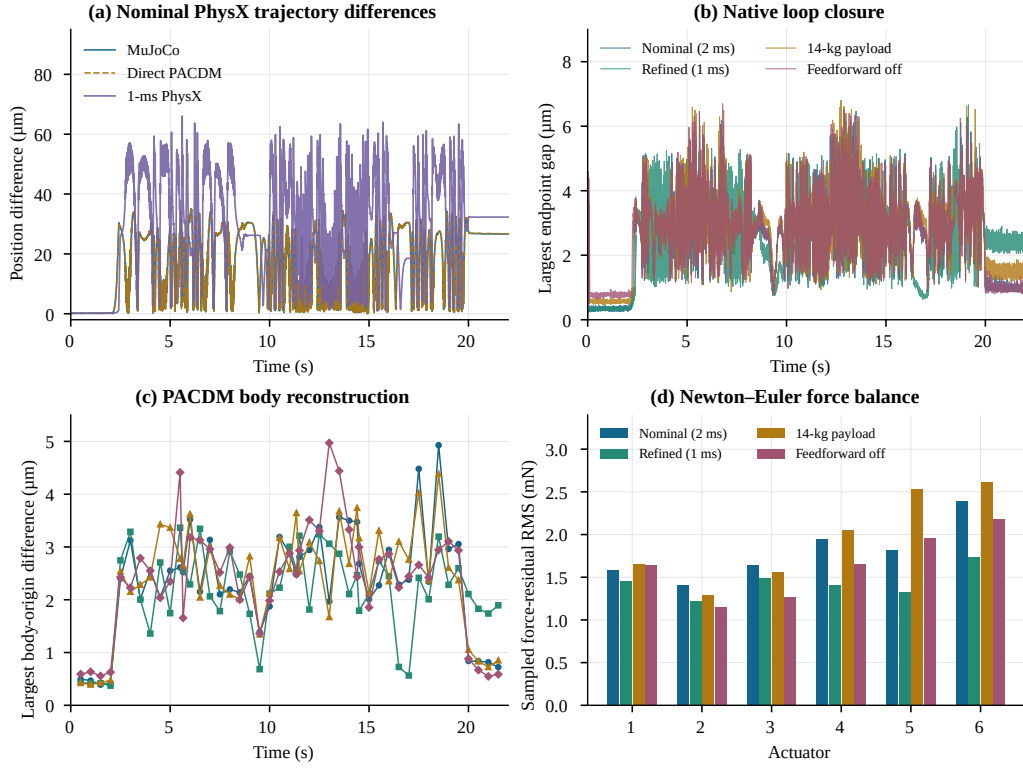


Fig. 11. Independent Stewart PhysX validation. (a) Position differences from MuJoCo, direct PACDM–Pinocchio, and the refined PhysX run at matched timestamps. (b) Maximum native spherical-closure gaps. (c) Body-origin discrepancies following fresh assembly at 46 measured states per case. (d) Per-actuator RMS Newton–Euler force-balance residuals from Eq. (26). Panels (c,d) use measured native states and freshly evaluated PACDM mappings.

For Kangaroo, the maximum energy-ledger residual decreases from 0.04108 to 0.02050 J when the MuJoCo step is halved from 25 to 12.5 μ s, and from 0.20313 to 0.10207 J when the Pinocchio step is halved from 1 to 0.5 ms. Halving the PhysX step changes terminal pelvis position by 0.873 mm and total actuator work by 0.08361 J; doubling position iterations changes them by 0.03687 mm and 7.35×10^{-4} J. Stewart refinement appears in Fig. 6(a).

VII. CONCLUSION

RoboCompiler compiles a canonical mechanism graph of physical bodies, joints, attachment frames, inertias, and actuator ports into one interface for closure, motion, actuation, and dynamics. From a shared assembled configuration, it generates the tangent lift, the constraint-curvature correction, and virtual-work-dual actuator-port maps, so that controllers, reduced dynamics, and external simulators refer to the same assembly branch and actuator coordinates. On three closed-chain mechanisms, RoboCompiler’s constrained accelerations agree with a 50-digit hard-constraint reference to a maximum normalized discrepancy of 5.1×10^{-7} ; on Stewart and Go2 at four times the prescribed speed, the curvature correction lowers acceleration residuals by about ten orders of magnitude. Generated cycle-local evaluation reduces Kangaroo residual- and Jacobian evaluation time by 96.7% and the wall time of a complete closed-loop rollout by 66.8% while preserving the computed response, and models exported from the same mechanism graph execute in MuJoCo, Pinocchio, and Isaac

Sim/PhysX, including native contact, material transfer, and closed-chain landing. Future work will transfer the compiled interfaces to hardware, extend the reduction through mechanism singularities, and synthesize transition and reset laws for connection changes during motion.

REFERENCES

- [1] M. H. Shahna and J. Mattila, “Model-free generic robust control for servo-driven actuation mechanisms with layered insight into energy conversions,” in *2025 American Control Conference (ACC)*. IEEE, 2025, pp. 4331–4338.
- [2] M. Chignoli, J.-J. Slotine, P. M. Wensing, and S. Kim, “URDF+: An enhanced URDF for robots with kinematic loops,” in *Proc. IEEE-RAS Int. Conf. Humanoid Robots (Humanoids)*, Nancy, France, 2024, pp. 197–204.
- [3] V. Batto, L. De Matteis, and N. Mansard, “Extended URDF: Accounting for parallel mechanism in robot description,” 2025, arXiv:2504.04767v2.
- [4] F. Díaz Ledezma and S. Haddadin, “Machine learning-driven self-discovery of the robot body morphology,” *Science Robotics*, vol. 8, no. 85, p. eadh0972, 2023.
- [5] R. Bordinaba, L. Ros, and J. M. Porta, “A randomized kinodynamic planner for closed-chain robotic systems,” *IEEE Transactions on Robotics*, vol. 37, no. 1, pp. 99–115, 2021.
- [6] S. Kumar, K. A. von Szadkowski, A. Mueller, and F. Kirchner, “An analytical and modular software workbench for solving kinematics and dynamics of series-parallel hybrid robots,” *Journal of Mechanisms and Robotics*, vol. 12, no. 2, 2020, art. no. 021114.
- [7] S. Kumar and A. Mueller, “An analytical and modular software workbench for solving kinematics and dynamics of series-parallel hybrid robots,” in *Proc. ASME Int. Design Eng. Tech. Conf. Comput. Inf. Eng. Conf. (IDETC/CIE), 43rd Mechanisms and Robotics Conf.*, vol. 5A, Anaheim, CA, USA, 2019, art. no. V05AT07A054.
- [8] E. Mingo Hoffman, A. Curti, N. Miguel, S. K. Kothakota, A. Molina, A. Roig, and L. Marchionni, “Modeling and numerical analysis of

- Kangaroo lower body based on constrained dynamics of hybrid serial-parallel floating-base systems,” *Robotics and Autonomous Systems*, vol. 182, 2024, art. no. 104827.
- [9] M. Dastranj and J. Mattila, “Modular kinematic reduction of closed-chain mechanisms using path assembly and defect homotopy,” arXiv:2609.11338, 2026.
 - [10] M. Chignoli, N. Adrian, S. Kim, and P. M. Wensing, “A propagation perspective on recursive forward dynamics for systems with kinematic loops,” *IEEE Transactions on Robotics*, vol. 41, pp. 5584–5603, 2025.
 - [11] M. Chignoli, N. Adrian, and P. M. Wensing, “GRBDA: Generalized rigid-body dynamics algorithms,” https://github.com/ROAM-Lab-ND/generalized_rbda, commit e68a6ec (2026-06-18).
 - [12] D. J. Volpi and P. M. Wensing, “Adapting rigid-body dynamics derivatives for constraint embedding closed-chain models,” *IEEE Robotics and Automation Letters*, vol. 11, no. 9, pp. 10 784–10 791, 2026.
 - [13] J. Carpentier, G. Saurel, G. Buondonno, J. Mirabel, F. Lamiriaux, O. Stasse, and N. Mansard, “The Pinocchio C++ library: A fast and flexible implementation of rigid body dynamics algorithms and their analytical derivatives,” in *Proc. IEEE/SICE Int. Symp. System Integration (SII)*, Paris, France, 2019, pp. 614–619.
 - [14] A. S. Sathya and J. Carpentier, “Constrained articulated body algorithms for closed-loop mechanisms,” *IEEE Transactions on Robotics*, vol. 42, pp. 819–838, 2026.
 - [15] “Pinocchio v4.1.0,” <https://github.com/stack-of-tasks/pinocchio/releases/tag/v4.1.0>, 2026, released 2026-07-07 (tag commit 2ae7766).
 - [16] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
 - [17] “MuJoCo documentation: Changelog, Version 3.7.0 (April 14, 2026) and Version 3.14.0 (September 22, 2026),” <https://mujoco.readthedocs.io/en/stable/changelog.html>, accessed 2026-09-28.
 - [18] M. H. Shahna and J. Mattila, “Exponential auto-tuning fault-tolerant control of n degrees-of-freedom manipulators subject to torque constraints,” in *2024 IEEE 63rd Conference on Decision and Control (CDC)*. IEEE, 2024, pp. 7263–7270.
 - [19] A. Escande, N. Mansard, and P.-B. Wieber, “Hierarchical quadratic programming: Fast online humanoid-robot motion generation,” *The International Journal of Robotics Research*, vol. 33, no. 7, pp. 1006–1028, 2014.
 - [20] A. Albu-Schäffer and A. Sachtler, “Redundancy resolution at position level,” *IEEE Transactions on Robotics*, vol. 39, no. 6, pp. 4240–4261, 2023.
 - [21] A. Bylard, R. Bonalli, and M. Pavone, “Composable geometric motion policies using multi-task pullback bundle dynamical systems,” 2021, arXiv:2101.01297v2.
 - [22] L. De Matteis, V. Batto, J. Carpentier, and N. Mansard, “Optimal control of walkers with parallel actuation,” 2025, arXiv:2504.00642v1.
 - [23] M. H. Shahna, P. Mustalahti, and J. Mattila, “Anti-slip ai-driven model-free control with global exponential stability in skid-steering robots,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 6855–6862.
 - [24] R. Featherstone, *Rigid Body Dynamics Algorithms*. Springer, 2008.
 - [25] HuCeBot team, Inria/Loria, “mujoco_kangaroo_sim2sim,” https://github.com/hucebot/mujoco_kangaroo_sim2sim, 2025, commit c020b68, BSD 2-Clause license.
 - [26] K. Zakka, Y. Tassa, and MuJoCo Menagerie Contributors, “MuJoCo Menagerie: A collection of high-quality simulation models for MuJoCo,” https://github.com/google-deepmind/mujoco_menagerie, 2022, commits 822c2d8 (franka_emika_panda, Apache-2.0 license) and 367e3d9 (unitree_go2, BSD 3-Clause license).